

OverModRaise: Reducing Modulus Consumption of CKKS Bootstrapping

Jihwan Kim¹ , Jung Hee Cheon^{1,2}  and Yongdong Yeo¹ 

¹ Seoul National University, Seoul, Korea

² CryptoLab Inc., Seoul, Korea

Abstract. The Cheon-Kim-Kim-Song (CKKS) homomorphic encryption scheme is widely adopted for securely evaluating circuits over real numbers, such as those arising in privacy-preserving machine learning (PPML), because it efficiently supports approximate floating-point arithmetic of messages. A CKKS ciphertext has a finite level, which corresponds to the budget for how many multiplicative operations can be applied. Once these levels are consumed, the ciphertext must be refreshed through a bootstrapping procedure to restore its capacity for further computation. The CKKS bootstrapping procedure consists of four main steps: 1) ModRaise, which raises the ciphertext coefficient modulus; 2) C2S, which homomorphically evaluates the inverse-DFT (iDFT) to enable further operations on the ciphertext coefficients; 3) EvalMod, which homomorphically removes the unintended coefficients introduced by ModRaise and shifted to the message side by C2S; and 4) S2C, which homomorphically evaluates the DFT to map the ciphertext back from the iDFT domain. However, these bootstrapping procedures also consume a significant number of levels, leaving fewer levels after each bootstrapping.

In this work, we introduce three techniques—OverModRaise1, OverModRaise2, and Tuple-C2S/S2C—that target reductions in the modulus consumption of C2S/S2C among the CKKS bootstrapping procedures, without introducing substantial overhead or compromising security. By combining these techniques, our implementation demonstrates at most 41% throughput improvement compared to the state-of-the-art bootstrapping.

Keywords: Homomorphic encryption · bootstrapping · CKKS scheme

1 Introduction

Homomorphic Encryption (HE) allows us to perform operations on data while in an encrypted state. The Cheon-Kim-Kim-Song (CKKS) scheme [CKKS17] is one of the most widely used HE scheme, and it can efficiently handle real and complex data with floating-point arithmetic. This is achieved through a special DFT-based encoding structure that maps floating-point message vectors to integer-coefficient plaintext polynomials, thereby distinguishing messages from plaintexts in the CKKS scheme. In CKKS, the ciphertext has a certain modulus, which decreases as we proceed with homomorphic operations. In particular, homomorphic multiplication consumes a portion of the ciphertext modulus, as it requires a procedure called rescale.

When multiplying two ciphertexts, or a ciphertext with a plaintext, the modulus decreases by the scaling factor. We call each amount of modulus consumed by one multiplication as one level. For instance, suppose the scaling factor is 2^{50} and the current modulus is 2^{1000} . In this case, we say that 20 levels remain. After one multiplication, the

E-mail: aqua4689@snu.ac.kr (Jihwan Kim), jhcheon@snu.ac.kr (Jung Hee Cheon), yongdong@snu.ac.kr (Yongdong Yeo)



remaining number of levels becomes 19. This scenario is referred to as leveled CKKS, first introduced in [CKKS17].

We use all the levels at some point, and the ciphertext modulus becomes too small to perform further operations. One may think that we need to set the fresh ciphertext modulus (a.k.a. top-level ciphertext modulus) right after the encryption to be big enough to avoid this situation. However, this approach is not desirable, as the Ring Learning With Errors (RLWE) samples used in the CKKS scheme provide weaker security as the modulus increases. We emphasize that there exists an upper bound on the secure modulus for both the key and the ciphertext, and that this maximum key modulus must not be exceeded during any operation on the ciphertext to ensure security. For this reason, leveled HE, including CKKS, necessitates a bootstrapping procedure [CHK⁺18], which makes the ciphertext modulus bigger so further homomorphic operations can be performed on the bootstrapped ciphertext.

In bootstrapping, we first raise the input ciphertext modulus to the maximum ciphertext modulus by the step called **ModRaise**. Consequently, the decryption of the output ciphertext of **ModRaise** becomes $m(X) + q \cdot I(X)$ over the ciphertext polynomial space, which differs from the original plaintext $m(X)$ by a multiple of q , for some polynomial $I(X)$. The difference $q \cdot I(X)$ in the coefficients can be removed by homomorphic operations acting on the message space. Specifically, by homomorphically evaluating the inverse-DFT (iDFT), the coefficients of $m(X) + q \cdot I(X)$ are transferred to the message (a.k.a. slot) side, a procedure referred to as Coefficient-to-Slot (C2S). Then, in the **EvalMod** step, homomorphic modular q reduction is applied, removing the $q \cdot I(X)$ terms, and the coefficients of $m(X)$ in the slot side are transferred back to the coefficient side via Slot-to-Coefficient (S2C), which corresponds to a homomorphic DFT.

As the bootstrapping itself uses some modulus, the output ciphertext modulus after bootstrapping is smaller than the maximum ciphertext modulus. Therefore, reducing the modulus consumption of bootstrapping is important since the more modulus of the output ciphertext, the more homomorphic operation capacity per bootstrapping. Hence, reducing the modulus consumption of bootstrapping can enhance the overall performance of the CKKS scheme.

This work provides three techniques to reduce the modulus consumption of C2S and S2C within bootstrapping. In general, each of C2S and S2C steps consumes 2 to 4 levels, depending on the decomposition of their (i)DFT matrices to balance latency and level consumption. Our proposals focus on reducing the level consumption when the decomposition of the C2S and S2C matrices is kept in the usual form, enabling them to be applied to existing schemes without incurring significant bootstrapping latency overhead.

Conversely, the techniques can also be used to decompose the C2S and S2C matrices into more submatrices than in the typical setting. In this case, level consumption is preserved while the computational cost can be reduced due to the increased sparsity of the matrices. In summary, our methods enable a trade-off between computational cost and level consumption. In the analysis presented in this paper, we focus primarily on reducing the level consumption.

1.1 Our Contribution

In this paper, we propose three different techniques to reduce the modulus consumption during the standard CKKS bootstrapping¹:

- We analyze **OverModRaise**², which achieves a 1-level gain over standard CKKS

¹The CKKS bootstrapping that utilizes **ModRaise**, **C2S**, **EvalMod**, **S2C** as its sub-algorithms.

²This technique has been informally known to some as a kind of folklore that the modulus could be raised beyond the ciphertext maximum, avoiding key-switching. However, to the best of our knowledge, no documentation or implementation exists that applies this idea by altering the key-switching order in C2S of CKKS bootstrapping.

bootstrapping by increasing the input ciphertext modulus more than the maximum ciphertext modulus in the **ModRaise** step. This is done by reordering the rotation and constant multiplication operations in the first linear transformation of **C2S**.

- We propose **OverModRaise2**, which, like **OverModRaise1**, leverages an extra modulus beyond the maximum ciphertext modulus. It discards a portion of the less significant bits of the ciphertext prior to **C2S** and subsequently restores the message scale after **C2S**. While this process introduces an additional rescaling error, we present a method to control this error such that it remains smaller than the inherent **C2S** error. When combined with **OverModRaise1**, the modulus advantage can be approximately equal to the size of the base prime in the CKKS scheme.
- We actualize a way to apply Tuple-CKKS on the linear transformations of CKKS bootstrapping (*Tuple-C2S/S2C*) and observe that it can give a similar or better modulus advantage compared to Tuple-CKKS on ciphertext-ciphertext multiplication. We also provide a detailed analysis of applying Tuple-CKKS to homomorphic rotations, constant multiplications, and linear transformations, which are not analyzed in [CKKS23]. This approach achieves about 2-level advantage over ordinary bootstrapping.
- We provide the analysis for putting all three of our methods to further reduce the modulus consumption of CKKS bootstrapping. We quantify the modulus savings contributed by each individual technique. Finally, our implementation demonstrates at most 41% throughput improvement over the state-of-the-art CKKS bootstrapping method, EvalRound+ [SSKM25].

1.2 Technical Overview

The first method, **OverModRaise1**, raises the modulus one level higher than the maximum ciphertext modulus used in the standard CKKS bootstrapping **ModRaise**. In this setting, the subsequent **C2S** step performs a homomorphic multiplication by the iDFT matrix [HS14], which consists of a homomorphic rotation followed by plaintext-ciphertext multiplication. The initial homomorphic rotation operates over modulus PQ_{in} , where Q_{in} is the modulus of the input ciphertext. However, in **C2S** after **OverModRaise1**, Q_{in} is larger than in the standard approach, resulting in a security vulnerability due to the increased modulus.

To address this issue, we adopt a simple technique in **OverModRaise1**: switching the order of the rotation and the plaintext-ciphertext multiplication. Specifically, performing the plaintext-ciphertext multiplication first reduces the ciphertext modulus before rotation is applied. This ensures that **C2S** and all subsequent operations are carried out securely over the modulus PQ , which the conventional bootstrapping uses.

We propose the second method, **OverModRaise2**, which further increases the modulus advantage. In CKKS bootstrapping, the precision of the iDFT matrix encodings used in **C2S** is lower than that of the input ciphertext. **OverModRaise2** reduces the wasted ciphertext precision and the corresponding modulus consumption resulting from this mismatch. To achieve this, we first raise the modulus in **ModRaise**, similar to **OverModRaise1**, and immediately rescale the ciphertext. Then, we apply **C2S** to the ciphertext with reduced precision. After **C2S**, we use inverse rescale to restore the message scale, thereby obtaining an advantage in modulus consumption. Consequently, the modulus used in the immediate rescale after modulus raising—which equals the amount required in the inverse rescale—is effectively saved over the entire bootstrapping procedure. Since the EvalRound algorithm already uses lower precision for the iDFT matrix diagonal encodings in **C2S**, this method is particularly effective in that context.

The third technique is **Tuple-C2S** and **Tuple-S2C**, which apply Tuple-CKKS [CKKS23] to **C2S** and **S2C**. Tuple-CKKS is a method for reducing modulus consumption in ciphertext-

ciphertext multiplications. While prior work focused on such multiplications, we extend the idea to ciphertext-plaintext multiplications used in linear transformations, including C2S and S2C. Tuple-CKKS decomposes a ciphertext into two parts via Euclidean division; we refer to the remainder as the *lower part*, which dominates the error in the scheme. The size of the lower part grows when adding decomposed ciphertexts, which is relevant since homomorphic linear transformations—such as C2S and S2C—involve summing many terms. This raises concerns about error growth. However, ciphertext-plaintext multiplication, which appears in linear transformations, incurs less error than ciphertext-ciphertext multiplication. As both negative (summation error) and positive (lower multiplication error) factors interplay in linear transformations, we argue that applying Tuple-CKKS to C2S and S2C provides a modulus advantage comparable to or better than that in ciphertext-ciphertext multiplication. We support this claim with a detailed error analysis in Section 4 and Appendix C.

1.3 Related Works

CKKS Bootstrapping. CKKS bootstrapping was first proposed in [CHK⁺18], which enables using the earlier version of the leveled CKKS scheme as a fully homomorphic encryption scheme. The baseline idea of CKKS bootstrapping can be understood as procedures to increase the modulus and then recover the message. The CKKS bootstrapping method passed through several improvements by earlier works.

Our work focuses on C2S and S2C part of CKKS bootstrapping, and there were other works introducing methods to make C2S and S2C better which are homomorphic DFT and iDFT. In [HHC19], authors decomposed C2S and S2C matrix into sparse matrices to reduce the computational cost. They first decomposed into around 15 matrices using the Cooley-Tukey algorithm and combined them into 3 or 4 matrices. In [BMTH21], authors proposed double-hoisting, which can reduce the number of approximate base changes for RNS-CKKS while doing C2S and S2C. In [KLK⁺22], a strategy to minimize the key size for homomorphic DFT and iDFT was introduced.

There are also several homomorphic linear transformation optimizations [HS18, HS21] with the BGV scheme, which are also utilized in later works for CKKS bootstrapping. As CKKS bootstrapping requires evaluating the approximate modular reduction function by polynomials, there is a line of works [CHK⁺18, CCS19, LLL⁺21, JM20, LLK⁺22] to advance the EvalMod procedure with numerical analysis.

Reducing Modulus Consumption. Tuple-CKKS [CCKS23] reduces modulus consumption of homomorphic multiplication. As we use homomorphic multiplication in EvalMod, their work can be used to reduce modulus consumption of EvalMod. Our work suggests using Tuple-CKKS also for linear transformation, which can reduce modulus consumption of C2S and S2C.

EvalRound [KPK⁺22] reduced the modulus consumption of C2S by using EvalRound instead of EvalMod. Using the property that EvalRound is more robust to the input error, they reduced the modulus consumption of C2S. When combined with our OverModRaise2, this leads to an even larger modulus advantage.

In EvalRound+ [SSKM25], authors resolved the disadvantage of EvalRound, particularly regarding the larger S2C scaling factor, by introducing an additional parallel C2S process. They named this enhanced algorithm as EvalRound+. The notable change of the CKKS bootstrapping algorithm in their paper was extra parallel C2S, while others were just simple reductions of the C2S and S2C scaling factors. As extra parallel C2S does not cause problems with our method, their methods and our methods are orthogonal and can be successfully combined. If we combine the method of EvalRound+ and the methods in our paper, the modulus advantage becomes greater. We provided the parameter set and experimental result in Section 6.

A recent study, SHIP [CHKS25], proposed a new bootstrapping algorithm that replaces the traditional EvalMod and C2S steps with a blind rotation technique. Thanks to the new bootstrapping structure, it becomes possible to trade off the modulus consumption and latency required for bootstrapping, enabling bootstrapping even with a ring dimension of $N = 2^{13}$. However, for parameters with larger ring dimensions (e.g., $> 2^{15}$), the throughput improvement becomes limited, and the approach becomes less advantageous in environments where parallelization is not available. On the other hand, our method can be integrated into the standard bootstrapping pipeline without significantly increasing latency, however, OverModRaise1 (Section 3.1) can also be combined with SHIP to reduce modulus consumption further.

1.4 Road Map

Section 2 reviews the CKKS scheme and introduces key prior works. Section 3 presents our new concept OverModRaise, which performs ModRaise into a modulus larger than the maximum modulus. Section 3.1 describes the simplest method, OverModRaise1, which achieves OverModRaise securely by reordering operations in the first linear transformation of C2S. Section 3.2 explains the second method, OverModRaise2, which instead performs OverModRaise via ciphertext downscaling. Section 4 adapts Tuple-CKKS [CCKS23] for C2S and S2C and analyzes the associated parameter settings. In Section 5, we combine all three methods (OverModRaise1, OverModRaise2, and Tuple-C2S/S2C) to further improve level efficiency and provide concrete parameter sets. Finally, Section 6 presents experimental results demonstrating a 41% improvement in throughput over the state-of-the-art.

2 Preliminaries

2.1 Notations

In this paper, $[\cdot]_p$ denotes the modulo reduction by p , which yields a value in the range $[-p/2, p/2]$. The notation $\langle \cdot, \cdot \rangle$ represents the inner product, although, for simplicity, we occasionally use $\cdot$ to indicate the same operation. Let $R = \mathbb{Z}[X]/(X^N + 1)$ and $R_Q = \mathbb{Z}_Q[X]/(X^N + 1)$. We denote the ciphertext modulus chain as $Q_0 < \dots < Q_L$. In particular, we define $q = Q_0$ as the smallest possible modulus in the CKKS scheme and $Q_{\max} = Q_L$ as the maximum ciphertext modulus such that $PQ_{\max}$ represents the maximum modulus that meets the security level. Here, P is a special modulus used for Key-Switching, which will be described later. For simplicity, we often refer to $Q_{\max}$ as the maximum modulus under the security condition.

2.2 CKKS Scheme

The CKKS scheme introduced in [CKKS17] is a leveled homomorphic encryption scheme that supports approximate arithmetic over vectors of complex (and real) numbers. A vector of messages in $\mathbb{C}^{N/2}$ is encoded as a plaintext polynomial in $R = \mathbb{Z}[X]/(X^N + 1)$ via the canonical embedding $\text{can} : \mathbb{R}[X]/(X^N + 1) \rightarrow \mathbb{C}^{N/2}$, defined by $\text{can}(m(X)) = (m(\zeta^{5^j}))_{0 \leq j < N/2}$, where ζ is a primitive $2N$ -th root of unity. Given a scaling factor Δ , the encoding and decoding procedures are defined as follows:

- **Encoding** : For a message $z \in \mathbb{C}^{N/2}$, encoding is defined as $\text{Ecd}(z, \Delta) = \lfloor \Delta \cdot \text{can}^{-1}(z) \rfloor$. The parameter Δ in this definition is called the scaling factor. Throughout the paper, we often use *slot* for each component of the message z and *coefficient* for each coefficient of the polynomial $\text{Ecd}(z, \Delta)$.
- **Decoding** : For a plaintext $m(X) \in R$, decoding is defined as $\text{Dcd}(m(X), \Delta) = \text{can}(\Delta^{-1} \cdot m(X))$.

A plaintext polynomial in R can be encrypted into a ciphertext in $R_Q \times R_Q$, where decryption maps back to R . Unless otherwise specified, Q denotes an arbitrary ciphertext modulus, and Q_l a level- l modulus in the chain. For a standard deviation σ , $\mathcal{DG}(\sigma^2)$ denotes the discrete Gaussian distribution over $\mathbb{Z}$ with variance σ^2 . Similarly, $\mathcal{ZO}(0.5)$ denotes the centered ternary distribution with probabilities $(1/4, 1/2, 1/4)$ over $1, 0, -1$.

- **Key generation** : We will sample the secret key $\mathbf{sk}$, public key $\mathbf{pk}$, and switching key $\mathbf{swk}$. First, we sample $s \in R$ such that its coefficients are in $-1, 0, 1$ and satisfy the Hamming weight condition h . The secret key is then defined as $\mathbf{sk} = (1, s)$. Next, we sample a_0 uniformly from R_{Q_L} and $e \leftarrow \mathcal{DG}(\sigma^2)$. Using these, the public key is defined as $\mathbf{pk} = (-a_0 \cdot s + e, a_0) \in R_{Q_L}^2$. For the switching key, we sample a_1 uniformly from R_{PQ_L} and $e' \leftarrow \mathcal{DG}(\sigma^2)$. We then define the switching key as $\mathbf{swk}_{s_1 \rightarrow s_2} = (-a_1 \cdot s_2 + e' + P \cdot s_1, a_1) \in R_{PQ_L}^2$.
- **Encryption**: Given a plaintext $m(X) \in R$, sample $v \leftarrow \mathcal{ZO}(0.5)$ and $e_0, e_1 \leftarrow \mathcal{DG}(\sigma^2)$. The ciphertext is computed as $\text{Enc}(m) = v \cdot \mathbf{pk} + (m + e_0, e_1)$.
- **Decryption**: For a ciphertext $\mathbf{ct} = (b, a) \in R_Q^2$, the decryption is computed as $\text{Dec}(\mathbf{ct}) = \lfloor b + a \cdot s \rfloor_Q$.

Next, we describe the basic homomorphic operations of the CKKS scheme. Homomorphic multiplications involve the rescaling (RS) procedure, which reduces the scaling factor of the ciphertext. We also employ the inverse rescaling (InvRS) procedure [CCK⁺24], which is the reverse operation of RS. Inverse rescaling can be seen as introducing a free modulus, but it increases the error, so it is used only in cases where the error growth can be safely tolerated.

- **RS(ct)**: For $\mathbf{ct} \in R_Q^2$, rescaling by q is defined as $\text{RS}_q(\mathbf{ct}) = \lfloor \mathbf{ct}/q \rfloor \in R_{Q/q}^2$.
- **InvRS(ct)**: For $\mathbf{ct} \in R_Q^2$, inverse rescaling by q is defined as $\mathbf{ct} \in R_Q^2$ by $\text{InvRS}_q(\mathbf{ct}) = q \cdot \mathbf{ct} \in R_{qQ}^2$.

Next, we explain Key-Switching, which is used in homomorphic multiplication and rotation. Key-Switching converts the secret key of a ciphertext while preserving the encrypted message.

- **Key-Switching $_{s_1 \rightarrow s_2}(\mathbf{swk}_{s_1 \rightarrow s_2}, \mathbf{ct})$** : For a ciphertext $\mathbf{ct} = (b, a) \in R_Q^2$ encrypted by a secret key $\mathbf{sk}_1 = (1, s_1)$, let $\mathbf{sk}_2 = (1, s_2)$ be a target secret key. To perform key switching, we use the switching key $\mathbf{swk}_{s_1 \rightarrow s_2}$ which is reduced modulo PQ prior to use. The key switching operation from the secret $\mathbf{sk}_1$ to the secret $\mathbf{sk}_2$ is defined by $\text{Key-Switching}_{s_1 \rightarrow s_2}(\mathbf{ct}) = (b, 0) + \lfloor a \cdot \mathbf{swk}_{s_1 \rightarrow s_2}/P \rfloor$.

One can check that $\text{Dec}_{\mathbf{sk}_2}(\text{Key-Switching}_{s_1 \rightarrow s_2}(\mathbf{ct})) \approx \text{Dec}_{\mathbf{sk}_1}(\mathbf{ct})$ with a small error. During the procedure, it is important to note that the switching key operates over modulus PQ , which must be smaller than the maximum modulus $PQ_{\max}$ to ensure security.

- **Add(ct₁, ct₂)**: For $\mathbf{ct}_1, \mathbf{ct}_2 \in R_{Q_l}^2$, the operation is defined by $\text{Add}(\mathbf{ct}_1, \mathbf{ct}_2) = \mathbf{ct}_1 + \mathbf{ct}_2$.
- **PtMult(pt, ct)**: Let $\mathbf{pt} \in R_{Q_l}$ and $\mathbf{ct} = (b, a) \in R_{Q_l}^2$ be a plaintext and a ciphertext with scaling factors Δ_A and Δ , respectively. First, compute $(b \cdot \mathbf{pt}, a \cdot \mathbf{pt})$ over $R_{Q_l}^2$. To restore the scaling factor from $\Delta_A \Delta$ to Δ while reducing the error, perform rescaling as $\text{PtMult}(\mathbf{pt}, \mathbf{ct}) = \text{RS}_{\Delta_A}((b \cdot \mathbf{pt}, a \cdot \mathbf{pt}))$.

- **Mult**(ct_1, ct_2): For two ciphertexts $\text{ct}_1 = (b_1, a_1)$ and $\text{ct}_2 = (b_2, a_2)$ in $R_{Q_l}^2$ with scaling factor Δ , multiplication in CKKS (Mult) is performed by sequentially applying three steps: tensor, relinearization, and rescaling. Tensor is defined as $\text{Tensor}(\text{ct}_1, \text{ct}_2) = (b_1 a_1, b_1 a_2 + b_2 a_1, b_2 a_2) \in R_{Q_l}^3$. Given the tensor output $\text{ct}_T = (t_1, t_2, t_3)$, relinearization is defined by $\text{Relin}(\text{ct}_T) = (t_1, t_2) + \lfloor t_3 \cdot \text{swk}_{s^2 \rightarrow s} / P \rfloor$. For the final step, rescale by Δ , RS_Δ , is applied.
- **Rot_k**(ct): Let $\text{ct} = (b(X), a(X))$ be a ciphertext encrypting $z = (z_1, \dots, z_{N/2}) \in \mathbb{C}^{N/2}$ under a secret key $\text{sk} = (1, s)$. The CKKS rotation Rot_k outputs a ciphertext where the message is $\rho(z; k) = (z_{k+1}, \dots, z_{N/2}, z_1, \dots, z_k)$. First, perform the automorphism Aut_j to the ciphertext ct defined by $\text{Aut}_j(\text{ct}) = (b(X^{5^k}), a(X^{5^k}))$. Since the resulting $\text{Aut}_j(\text{ct})$ becomes a ciphertext encrypting the message $\rho(z; k)$ with respect to the secret key $(1, s(X^{5^k}))$, apply $\text{Key-Switching}_{s(X^{5^k}) \rightarrow s}$ to $\text{Aut}_j(\text{ct})$.

For more details on the CKKS scheme, we refer to [CKKS17].

2.3 CKKS Bootstrapping

CKKS bootstrapping consists of sequentially evaluating the following four steps:

- **ModRaise** (Modulus Raise) : We begin CKKS bootstrapping with the ciphertext modulus set to the smallest modulus, q . Let $m(X)$ be the decrypted message $m(X) = \lfloor \langle \text{ct}, \text{sk} \rangle \rfloor_q$ and $t(X)$ be $t(X) = \langle \text{ct}, \text{sk} \rangle \pmod{X^N + 1}$. Then, for some small $I(X)$, we have $t = m + qI$. Next, we raise the modulus to the maximum ciphertext modulus $Q_{\max}$, where $Q_{\max}$ is sufficiently larger than q and exceeds the coefficients of $t(X)$. This ensures that $\lfloor \langle \text{ct}, \text{sk} \rangle \rfloor_{Q_{\max}} = t(X)$ holds. Consequently, our current message under modulus $Q_{\max}$ becomes $t(X) = m(X) + qI(X)$.
- **C2S** (Coefficient-to-Slot) : To recover the message $m(X)$ from $t(X)$, we need to homomorphically pack the coefficients of $t(X)$ into the slots. Through the homomorphic iDFT matrix multiplication, this can be done to obtain $\frac{1}{\Delta}t$ in the slots. For efficiency, the iDFT matrix is typically decomposed into 3 or 4 sparse matrices using the FFT method, and these matrices are then multiplied as described in [HHC19].
- **EvalMod** (Modular Reduction) : Next, we convert $\frac{1}{\Delta}t$ to $\frac{1}{\Delta}m$. This conversion uses homomorphic polynomial evaluation of approximated function $f(x) = \frac{1}{\Delta}[\Delta x]_q$ on the slots $\frac{1}{\Delta}$. In this case, the mod 1 function ($[\cdot]_1$) is approximated into a polynomial as a scaled version of $[\cdot]_q$. Since q is significantly larger (e.g., 2^{10} times) than m in typical CKKS parameters, the mod 1 function can be well-approximated by a sine function within a restricted range near points of the form $\frac{1}{\Delta}qI$.
- **S2C** (Slot-to-Coefficient) : Now, we have $\frac{1}{\Delta}m$ in the slots, but the ultimate goal is to retrieve $m(X)$ in the coefficients. To achieve this, we need to homomorphically DFT transform the slots into $\frac{1}{\Delta}\text{can}(m)$, which corresponds to the reverse procedure of C2S. Similar to C2S, we decompose the DFT matrix into 3 or 4 sparse matrices for efficiency and perform matrix multiplications. [HHC19]

The bootstrapping procedure described above is referred to as C2S-first bootstrapping because the C2S operation is performed before S2C. Alternatively, there is also an S2C-first bootstrapping version, where the execution order is S2C, C2S, ModRaise, and EvalMod. For further details, please refer to [BCC⁺22].

2.4 Linear Transformation in CKKS

Let $\text{ct} = (b(X), a(X))$ be a ciphertext with a message $z \in \mathbb{C}^{N/2}$. For a matrix $A \in \mathbb{C}^{N/2 \times \mathbb{C}^{N/2}}$, the goal is to obtain a ciphertext with the message $A \cdot z$. Define j -th diagonal vector $u_j = (A_{k \bmod N/2, j+k \bmod N/2})_{k=0}^{N/2} \in \mathbb{C}^{N/2}$ of the matrix A . In [HS14], the following formula was presented. Here, $\odot$ represents the Hadamard product between two vectors, and $\rho(z; j)$ was previously defined in the explanation of Rot (Section 2.2).

$$A \cdot z = \sum_{0 \leq j < N/2} (u_j \odot \rho(z; j)).$$

So we can define linear transformation with matrix A on ciphertext ct as below.

$$A \cdot \text{ct} = \sum_{0 \leq j < N/2} \text{PtMult}(\text{Ecd}(u_j, \Delta_A), \text{Rot}_j(\text{ct})). \quad (2.1)$$

With the Baby-Step Giant-Step (BSGS) style for N_1 and N_2 satisfying $N_1 \cdot N_2 = N/2$, the formula above is equivalent to

$$A \cdot \text{ct} = \sum_{0 \leq j < N_2} \text{Rot}_{N_1 \cdot j} \left(\sum_{0 \leq i < N_1} \text{PtMult}(\text{Ecd}(\rho(u_{N_1 \cdot j+i}; -N_1 \cdot j), \Delta_A), \text{Rot}_i(\text{ct})) \right). \quad (2.2)$$

If we set $N_1 \approx N_2$, it can reduce the number of Rot from $O(N)$ to $O(\sqrt{N})$.

2.5 EvalRound Algorithm

The EvalRound algorithm [KPK⁺22] modifies CKKS bootstrapping to improve modulus efficiency by replacing the EvalMod step with a more error-resilient EvalRound step, defined as $\text{EvalRound}(x) = x - \text{EvalMod}(x)$. Unlike EvalMod , which extracts the message m , EvalRound isolates the qI term, enabling recovery of m by subtraction. This step mitigates the effect of additive noise in the input, making it more robust than EvalMod . As a result, EvalRound allows the use of smaller scaling factors in the C2S step, reducing modulus consumption. The following diagram, adapted from [KPK⁺22], summarizes the core idea of the EvalRound procedure. For a complete description, we refer to [KPK⁺22].

$$\begin{aligned} \text{pt} &\xrightarrow{\text{ModRaise}} \text{pt} + qI \xrightarrow{\text{C2S}} (\text{pt} + qI + e)^* \\ &\xrightarrow{\text{EvalRound}} (qI)^* \xrightarrow{\text{S2C}} qI \xrightarrow{\text{Subtract}} \text{pt}. \end{aligned}$$

2.6 Tuple-CKKS

The Tuple-CKKS algorithm [CKKS23] is a method designed to reduce the modulus consumption of homomorphic multiplication. The main concept behind Tuple-CKKS is to decompose the ciphertext by performing Euclidean division on it. In the Tuple-CKKS paper, the authors initially discussed dividing the ciphertext into 2 tuple and then generalized the approach to dividing it into k tuple. For simplicity, in this paper, we will focus exclusively on the 2-tuple version and refer to it simply as Tuple-CKKS.

Definition 1 (Ciphertext Euclidean Division). Let $\text{ct} = (b, a) \in R_Q^2$ be the ciphertext, and let q be a divisor of Q . Then, the remainder of ct is defined as

$$\text{Rem}_q(\text{ct}) = ([b]_q, [a]_q) \in R^2.$$

The quotient of ct is defined as

$$\text{Quo}_q(\text{ct}) = RS_q(\text{ct}) = \frac{\text{ct} - \text{Rem}_q(\text{ct})}{q} \in R_{Q/q}^2.$$

Definition 2 (Decomposition and Recombination). Let $\text{ct} \in R_{Q_l}^2$ be the ciphertext, and let $Q'_1 = Q_l q_{\text{div}}$. The decomposition of ct is defined as

$$\text{DCP}_{q_{\text{div}}}(\text{ct}) = (\text{Quo}_{q_{\text{div}}}(\text{ct}), \text{Rem}_{q_{\text{div}}}(\text{ct})) \in R_{Q_l}^2 \times R_{Q_l}^2.$$

Conversely, for a given pair of decompositions $(\hat{\text{ct}}, \check{\text{ct}}) \in R_{Q_l}^2 \times R_{Q_l}^2$, the recombination of ct is defined as

$$\text{RCB}_{q_{\text{div}}}(\hat{\text{ct}}, \check{\text{ct}}) = q_{\text{div}} \cdot \hat{\text{ct}} + \check{\text{ct}} \in R_{Q_l}^2.$$

Lemma 1. Let q be a divisor of Q . Let $\text{ct} \in R_Q^2$ be the ciphertext, and $\text{sk} = (1, s)$ be the secret key with Hamming weight h . Let $m = [\text{ct} \cdot \text{sk}]_Q \in R$. Let $\check{m} = [m]_q \in R$ and $\hat{m} = (m - [m]_q)/q \in R$. Then the following holds for some $I \in R$ satisfying $\|I\|_\infty \leq (h+2)/2$:

$$\text{Quo}_q(\text{ct}) \cdot \text{sk} = \hat{m} + I \pmod{Q/q}.$$

$$\text{Rem}_q(\text{ct}) \cdot \text{sk} = \check{m} - qI.$$

In this paper, we will refer to the decryption of the remainder part as the *lower part*. From the theorem above, the initial bound of the message in the coefficients of the lower part will be $\frac{h+3}{2}q$. We will use this fact in the analysis of Section 4.

Definition 3 (Tuple Rescale). Let $CT = (\hat{\text{ct}}, \check{\text{ct}}) \in R_{Q_l}^2 \times R_{Q_l}^2$ be the decomposed ciphertext. Let $q_l = Q_l/Q_{l-1}$. The RS for CT is defined as

$$RS_{q_l}^2(CT) = (RS_{q_l}(\hat{\text{ct}}), RS_{q_l}(q_{\text{div}} \cdot \hat{\text{ct}} + \check{\text{ct}}) - q_{\text{div}} \cdot RS_{q_l}(\check{\text{ct}})) \in R_{Q_{l-1}}^2 \times R_{Q_{l-1}}^2.$$

Lemma 2. Let $CT \in R_{Q_l}^2 \times R_{Q_l}^2$ be the decomposed ciphertext. Let $q_l = Q_l/Q_{l-1}$. Let $\text{sk} = (1, s)$ be the secret key with Hamming weight h . The following holds:

$$\|[(\text{RCB}_{q_{\text{div}}}(RS_{q_l}^2(CT)) \cdot \text{sk})_{Q_{l-1}} - \frac{1}{q_l}[(\text{RCB}_{q_{\text{div}}}(CT)) \cdot \text{sk}]_{Q_l}]\|_\infty \leq \frac{h+1}{2}.$$

The more detailed analysis and the proofs of Lem. 1 and 2 above can be found in [CKKS23].

3 OverModRaise

In the conventional CKKS bootstrapping, **ModRaise** is a procedure that merely enlarges the modulus from the base (small) modulus q without performing any operation on the input ciphertext. Specifically, **ModRaise** increases the modulus to Q_{max} , the largest available ciphertext modulus, which is then gradually consumed by subsequent bootstrapping procedures. We note that the security parameter λ determines the maximum allowable public key modulus Q_{key} , and any RLWE sample (including ciphertexts and keys) remains secure as long as its modulus does not exceed Q_{key} . The secure modulus Q_{key} is factored into the product of the special modulus P and the maximum ciphertext modulus Q_{max} .

In this section, we introduce two methods that leverage the **OverModRaise** technique, which can also be combined in Section 5. For $Q_{\text{over}} > Q_{\text{max}}$, the **OverModRaise** technique raises the ciphertext modulus to Q_{over} during the **ModRaise** step, thereby reserving additional ciphertext modulus proportional to $Q_{\text{over}}/Q_{\text{max}}$ for use during and after CKKS bootstrapping. Although setting the ciphertext modulus larger than Q_{max} through **OverModRaise** does not itself compromise security, as long as it remains below PQ_{max} , a security issue arises if operations are performed at a modulus exceeding PQ_{max} , such as key switching with the enlarged modulus Q_{over} .

Definition 4 (OverModRaise). We begin CKKS bootstrapping with the ciphertext modulus set to the smallest modulus, q . Let $m(X)$ represent the decrypted message in terms of coefficients, which satisfies $m(X) = [\langle \text{ct}, \text{sk} \rangle]_q$. Define $t(X) = \langle \text{ct}, \text{sk} \rangle \pmod{X^N + 1}$. Then, for some small $I(X)$, the relationship $t = m + qI$ holds. Next, we raise the modulus to exceed the maximum modulus Q_{over} , where Q_{over} is larger than the conventional ModRaise modulus Q_{max} . This ensures that $[\langle \text{ct}, \text{sk} \rangle]_{Q_{\text{over}}} = t(X)$ holds. Consequently, our current message under modulus Q_{over} becomes $t(X) = m(X) + qI(X)$.

3.1 OverModRaise1: Multiplication-first

In this section, we present OverModRaise1, the simplest method for leveraging OverModRaise. OverModRaise1 performs the remaining CKKS bootstrapping steps starting from C2S with the same modulus consumption as the conventional approach, but operates on a ciphertext whose modulus has been raised to Q_{over} instead of applying the standard ModRaise step. As a result, the modulus of the output ciphertext is larger by a factor of $Q_{\text{over}}/Q_{\text{max}}$ compared to that obtained from standard bootstrapping.

However, when the homomorphic linear transformation C2S is applied directly to a ciphertext output from OverModRaise, the first key switching on the ciphertext with modulus Q_{over} must be performed using a switching key at modulus PQ_{over} , which leads to the aforementioned security issue. This arises because the first operation of C2S on the input ciphertext is a rotation (Rot), performed prior to reducing the ciphertext level below Q_{max} . The *Multiplication-first* method mitigates the security issue by delaying the use of the switching key, achieved by switching the execution order of Rot and PtMult. Recall that the linear transformation over CKKS can be done as Equation 2.1.

$$A \cdot \text{ct} = \sum_{0 \leq j < N/2} \text{PtMult}(\text{Ecd}(u_j, \Delta_A), \text{Rot}_j(\text{ct})).$$

In the formula above, the order of operation is Rot and then PtMult. Here the formula below outputs the same result as the formula above and applies operations in different order.

$$A \cdot \text{ct} = \sum_{0 \leq j < N/2} \text{Rot}_j(\text{PtMult}(\text{Ecd}(\rho(u_j; -j), \Delta_A), \text{ct})).$$

Here we apply the PtMult first and then apply the Rot. Since PtMult contains no operations over PQ_{max} , no security issue occurs.

In particular, by RS step included in PtMult, modulus goes down by Δ_{C2S} . So after the PtMult, modulus becomes $Q_{\text{over}}/\Delta_{\text{C2S}}$. Then we do Key-Switching in the modulus $Q_{\text{over}}/\Delta_{\text{C2S}}$, and the modulus of switching key is $PQ_{\text{over}}/\Delta_{\text{C2S}}$. So sufficient condition for secure computation is $PQ_{\text{over}}/\Delta_{\text{C2S}} \leq PQ_{\text{max}}$. Here we can set $Q_{\text{over}} = Q_{\text{max}}\Delta_{\text{C2S}}$, and this leads to extra modulus amount of Δ_{C2S} . The Figure 1 shows the change of modulus during OverModRaise and C2S. In the Figure 1, M_{C2S} is the total modulus consumption of C2S step. In ordinary CKKS bootstrapping modulus after C2S will be $Q_{\text{max}}/M_{\text{C2S}}$, but in our algorithm modulus after C2S is $Q_{\text{max}}\Delta_{\text{C2S}}/M_{\text{C2S}}$.

Matrix Decomposition. We note that the proposed method has a drawback in that it cannot utilize the BSGS (Baby-Step Giant-Step)-styled algorithm for the first linear transformation. As we do PtMult first, we have $N/2$ different inputs for Rot, so BSGS cannot reduce the number of Rot. In the usual setting, each decomposed C2S matrices would have the same amount of nonzero diagonals. So, not using the BSGS algorithm for the first linear transformation can make a disadvantage on computational cost. To reduce this disadvantage we can consider setting an unbalanced number of nonzero diagonals when we evaluate C2S as several decomposed linear transformations [HHC19]. By setting a small number of nonzero diagonals for the first linear transformation, we can reduce the

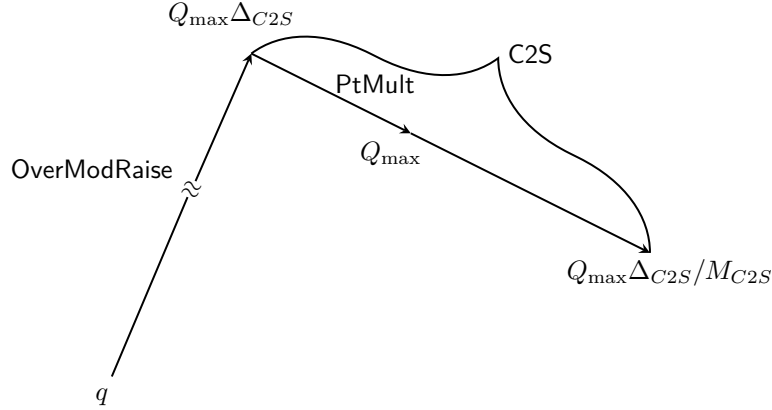


Figure 1: Multiplication-first OverModRaise1.

downside of not using the BSGS algorithm and still get a modulus advantage from our method.

To set an unbalanced number of nonzero diagonal decompositions of C2S matrix, we must look into the C2S decomposition structure. First, the C2S matrix with no decomposition has $N/2$ number of nonzero diagonals. If we decompose the C2S matrix using the Cooley-Tukey algorithm, we would have $\log_2(N/2)$ number of matrices, each having at most $2^{1+1} - 1 = 3$ nonzero diagonals. And if we combine k consecutive matrices from the Cooley-Tukey algorithm, then the combined matrix has around $2^{k+1} - 1$ number of nonzero diagonals. Let n_c be the final number of decomposed matrices after decomposing with the Cooley-Tukey algorithm and combining some consecutive matrices. And let us think of d as a fixed number for now. Then multiplication of the number of nonzero diagonals from each matrix will be around $2^{\log_2(N/2)+n_c}$. For ordinary CKKS bootstrapping, setting an equal amount of nonzero diagonals will be the optimal choice. So we would set $2^{(\log_2(N/2)+n_c)/n_c}$ amount of nonzero diagonals for each matrices. However, if we use the method in this section, setting an equal number of nonzero diagonals for the second to the n_c^{th} matrices and a smaller number of nonzero diagonals for the first matrix will be optimal. So we can set $2^{(\log_2(N/2)+n_c)/(n_c-1/2)}$ amount of nonzero diagonals for second to the n_c^{th} matrices, and set $2^{(\log_2(N/2)+n_c)/(2n_c-1)}$ amount of nonzero diagonals for first matrix. Here, we set the number of nonzero diagonals in the first matrix to be the square root of the number of nonzero diagonals in other matrices, but in real situations, the exponent of 2 needs to be a natural number, so it can be slightly different.

We provide some numerical examples. The first example is when $n_c = 4$ and the optimal decomposition for ordinary bootstrapping has $2^4, 2^4, 2^4, 2^5$ number of nonzero diagonals in four matrices. For this case setting $2^2, 2^5, 2^5, 2^5$ number of nonzero diagonals in four matrices is the optimal decomposition for our method. The second example is when $n_c = 3$ and optimal decomposition for ordinary bootstrapping has $2^5, 2^5, 2^6$ number of nonzero diagonals in three matrices. For this case setting $2^4, 2^6, 2^6$ number of nonzero diagonals in three matrices is the optimal decomposition for our method.

3.2 OverModRaise2: Ciphertext Downscaling

In this section, we propose another technique, *ciphertext downscale*, which also leverages OverModRaise of Definition 4. Contrary to the previous Multiplication-first technique in Section 3.1, which directly saves the additionally over-raised modulus Δ_{C2S} , the additional rescaling technique is based on the robustness of C2S error by performing an additional CKKS rescale procedure. Throughout this section, Δ_{C2S} is a scaling factor for C2S, and

n_c is the number of decomposed matrices in C2S.

Our algorithm is inspired by the question: Do we have to use the ciphertext with full precision during the C2S step? The answer is no, and there is no need to preserve the less significant bits of ciphertext, and we will drop them while applying C2S. To drop it, we perform RS by some factor Δ_S between the ModRaise step and C2S step. By inserting an additional rescale step, it is possible to ModRaise into the modulus $Q_{\max}\Delta_S$. This part is similar to the idea in Section 3.1, and it is possible to apply them together. We will discuss the combined case in Section 5.

After performing RS by Δ_S , we apply C2S on the rescaled ciphertext. Here, we will use a 1-bit larger Δ_{C2S} compared to the baseline bootstrapping algorithm for error control. After C2S we apply InvRS by Δ_S . By this, we recover the message scale before the EvalMod step.

Now we will compare the modulus before the EvalMod step. Let M_{C2S} be the modulus consumption of C2S in the baseline bootstrapping algorithm. For baseline bootstrapping algorithm, ModRaise makes the modulus to be $Q_{\max}$, and after C2S the modulus becomes $Q_{\max}/M_{C2S}$. For modified algorithm, we perform ModRaise into the modulus $Q_{\max}\Delta_S$, and perform RS by Δ_S and modulus becomes $Q_{\max}$. As we use 1 bit bigger scaling factor for C2S, modulus consumption of C2S will be $2^{n_c}M_{C2S}$. So after C2S the modulus becomes $Q_{\max}/(2^{n_c}M_{C2S})$. And by InvRS we get modulus $Q_{\max}\Delta_S/(2^{n_c}M_{C2S})$. This is $\Delta_S/2^{n_c}$ times bigger than the case of the baseline bootstrapping algorithm, so our modulus advantage is $\Delta_S/2^{n_c}$.

The Figure 2 shows the modulus change during OverModRaise and C2S. After the procedures in the picture, EvalMod will be applied.

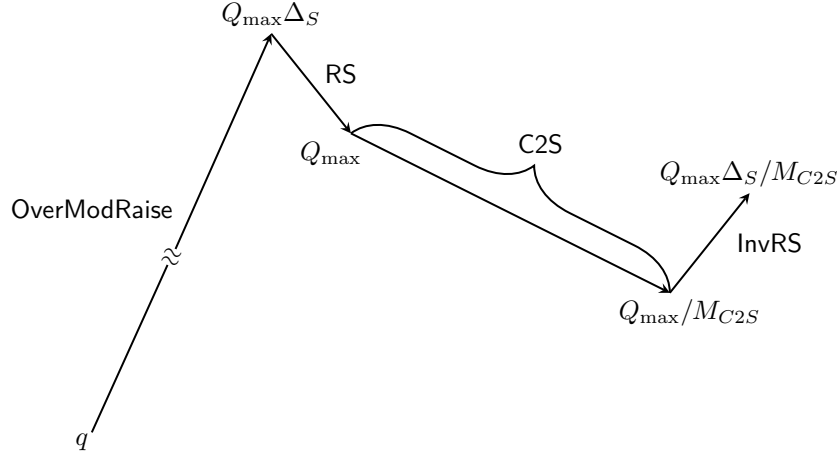


Figure 2: Ciphertext downscaling OverModRaise2.

The formulas below interpret the message change while applying our modified algorithm. $\mathbf{pt}$ is the message plaintext in polynomial space, e_{RS} is the rescale error in coefficients, e is the C2S error of baseline bootstrapping algorithm, and $*$ means that the content is in the Slot.

$$\begin{aligned}
 \mathbf{pt} &\xrightarrow{\text{ModRaise}} \mathbf{pt} + qI \\
 &\xrightarrow{\text{RS}} \frac{\mathbf{pt} + qI}{\Delta_S} + e_{RS} \xrightarrow{\text{C2S}} \left(\frac{\mathbf{pt} + qI}{\Delta_S} + e_{RS} + \frac{e}{\Delta_S} \right)^* \xrightarrow{\text{InvRS}} (\mathbf{pt} + qI + \Delta_S e_{RS} + e)^* \\
 &\xrightarrow{\text{EvalMod}} (\mathbf{pt} + \Delta_S e_{RS} + e)^* \xrightarrow{\text{S2C}} \mathbf{pt} + \Delta_S e_{RS} + e.
 \end{aligned}$$

3.2.1 Correctness

If the error $\Delta_{SE_{RS}} + e$ is acceptable, our algorithm will work correctly. In this section, we will find the condition that guarantees the correctness.

baseline algorithm applies **EvalMod** on $(\mathbf{pt} + qI + e(\Delta_{C_{2S}}))$ and our modified algorithm applies **EvalMod** on $(\mathbf{pt} + qI + \Delta_{SE_{RS}} + e(\Delta_{C_{2S}}))$. In the previous section, we wrote C_{2S} error as e for simplicity, but actually e depends on $\Delta_{C_{2S}}$. So from now we will write e as $e(\Delta_{C_{2S}})$, which is a random variable depending on $\Delta_{C_{2S}}$. Although other factors affect the e , we will only vary the $\Delta_{C_{2S}}$, so we can interpret e as $e(\Delta_{C_{2S}})$. We will show that the standard deviation of $e(\Delta_{C_{2S}})$ is inversely proportional to $\Delta_{C_{2S}}$. We formalize that fact in the lemma below.

Lemma 3. *Let the size of the input message, the size of q , and the scale of $\mathbf{pt}$ be fixed. Then C_{2S} error $\|e(\Delta_{C_{2S}})\|$ is a random variable depending on $\Delta_{C_{2S}}$. And its standard deviation is inversely proportional to $\Delta_{C_{2S}}$. In other words, for some constant C , the following holds.*

$$\mathbb{E}[\|e(\Delta_{C_{2S}})\|^2] = \frac{C}{(\Delta_{C_{2S}})^2}.$$

Proof. Let our initial message to be $z \in \mathbb{C}^{N/2}$ and decomposed C_{2S} matrices to be $A_1, \dots, A_d$. In the C_{2S} step, we calculate the $A_d \cdots A_1 z$. All the error terms have symmetric distribution [KPK⁺22], so we know that $e(\Delta_{C_{2S}})$ has zero mean.

As the decomposed C_{2S} matrices are conformal [HHC19], we prove a more general claim: for the multiplication of a row of conformal matrices, the standard deviation of the error scales inversely with $\Delta_{C_{2S}}$. The proof proceeds by induction.

Base case $d = 1$. The error arising from rounding during the encoding of matrix diagonals is the dominant error source in linear transformations [KPK⁺22], and we therefore focus on this error. Let $A := A_1$ be implemented via its diagonal decomposition $\{d_i\}_{i \in \mathcal{I}}$, and z_i denote z rotated by i slots. For each diagonal d_i , the encoder forms

$$\tilde{d}_i := \text{can}^{-1}(d_i) \in \mathbb{R}[X]/(X^N + 1), \quad \lfloor \Delta_{C_{2S}} \tilde{d}_i \rfloor \in R,$$

where $\lfloor \cdot \rfloor$ is elementwise rounding to the nearest integer. Define the elementwise rounding noise

$$\mathbf{r}_i := \lfloor \Delta_{C_{2S}} \tilde{d}_i \rfloor - \Delta_{C_{2S}} \tilde{d}_i \in \mathbb{R}[X]/(X^N + 1).$$

After decoding back, the error contributed by the i -th diagonal is

$$\varepsilon_i := \frac{1}{\Delta_{C_{2S}}} \text{can}(\mathbf{r}_i) \in \mathbb{C}^{N/2}.$$

When this diagonal is applied to the rotated message z_i , the resulting message-space error is the Hadamard product

$$e_i := \varepsilon_i \odot z_i.$$

Summing over all diagonals of A gives the total error

$$e(\Delta_{C_{2S}}) = \sum_{i \in \mathcal{I}} e_i = \frac{1}{\Delta_{C_{2S}}} \sum_{i \in \mathcal{I}} (\text{can}(\mathbf{r}_i) \odot z_i).$$

Finally, it leads to

$$\mathbb{E}[\|e(\Delta_{C_{2S}})\|_2^2] = \frac{1}{\Delta_{C_{2S}}^2} \mathbb{E} \left[\left\| \sum_{i \in \mathcal{I}} (\text{can}(\mathbf{r}_i) \odot z_i) \right\|_2^2 \right] = \frac{C}{\Delta_{C_{2S}}^2}.$$

Inductive step. First, let $e_{1:d-1}$ be the error generated when multiplying the first through $(d-1)$ -th matrices, and let e_d be the error generated when multiplying the d -th matrix. Then the total error satisfies

$$e_{\Delta_{C2S}} = A_d e_{1:d-1} + e_d.$$

Let $\mathcal{F}_{d-1}$ be the σ -algebra including all random variables determined up to the multiplication of the first through $(d-1)$ -th matrices. Since e_d arises from multiplying the rounding error (introduced by the diagonal encoding process of A_d) with the message at stage d , it is conditionally unbiased:

$$\mathbb{E}[e_d | \mathcal{F}_{d-1}] = 0.$$

Therefore,

$$\begin{aligned} \mathbb{E}[\langle A_d e_{1:d-1}, e_d \rangle] &= \mathbb{E}[\mathbb{E}[\langle A_d e_{1:d-1}, e_d \rangle | \mathcal{F}_{d-1}]] \\ &= \mathbb{E}[\langle A_d e_{1:d-1}, \mathbb{E}[e_d | \mathcal{F}_{d-1}] \rangle] \\ &= \mathbb{E}[\langle A_d e_{1:d-1}, 0 \rangle] = 0. \end{aligned}$$

Since the cross term is zero, we have

$$\begin{aligned} \mathbb{E}[\|e_{\Delta_{C2S}}\|_2^2] &= \mathbb{E}[\|A_d e_{1:d-1} + e_d\|_2^2] \\ &= \mathbb{E}[\|A_d e_{1:d-1}\|_2^2] + \mathbb{E}[\|e_d\|_2^2]. \end{aligned}$$

Since we assumed A_d to be conformal, we have

$$\mathbb{E}[\|A_d e_{1:d-1}\|_2^2] = \|A_d\|^2 \cdot \mathbb{E}[\|e_{1:d-1}\|_2^2],$$

so that

$$\mathbb{E}[\|e_{\Delta_{C2S}}\|_2^2] = \|A_d\|^2 \cdot \mathbb{E}[\|e_{1:d-1}\|_2^2] + \mathbb{E}[\|e_d\|_2^2].$$

By the induction hypothesis, both $\mathbb{E}[\|e_{1:d-1}\|_2^2]$ and $\mathbb{E}[\|e_d\|_2^2]$ are inversely proportional to $(\Delta_{C2S})^2$. Hence $\mathbb{E}[\|e_{\Delta_{C2S}}\|_2^2]$ is also inversely proportional to $(\Delta_{C2S})^2$. $\square$

Let's define the high probability bound $\|e(\Delta_{C2S})\|_\infty$ as $B(\Delta_{C2S})$. This means that $\|e(\Delta_{C2S})\|_\infty \leq B(\Delta_{C2S})$ holds for sufficiently high probability. Here $B(\Delta_{C2S})$ is the function with input Δ_{C2S} . In the lemma above, we saw that the standard deviation of the 2-norm of error is inversely proportional to Δ_{C2S} . As each slot has an identical distribution [KPK⁺22], the scale for the sup-norm of error is also inversely proportional to Δ_{C2S} , and for some C' , the formula $B(\Delta_{C2S}) = \frac{C'}{\Delta_{C2S}}$ will hold.

Now we define Δ_{C2S}^N as the scaling factor of C2S for the baseline algorithm. And let Δ_{C2S}^I be a scaling factor for the ordinary CKKS bootstrapping. Here, the baseline algorithm can be the ordinary CKKS bootstrapping or the EvalRound algorithm. If the baseline algorithm is the ordinary CKKS bootstrapping, $\Delta_{C2S}^N = \Delta_{C2S}^I$ holds. If the baseline algorithm is EvalRound algorithm, Δ_{C2S}^N is smaller than Δ_{C2S}^I . In our modified algorithm, we set Δ_{C2S} to be 1-bit larger than the baseline algorithm, so the scaling factor of C2S for our algorithm becomes $2 \cdot \Delta_{C2S}^N$. We note that $B(\Delta_{C2S}^N)$ is the bound of the C2S error e in the baseline algorithm. As the bound function is inversely proportional to the Δ_{C2S} , $B(2\Delta_{C2S}^N) = B(\Delta_{C2S}^N)/2$ will be the bound of C2S error e in our modified algorithm.

In the baseline algorithm, we can get an appropriate output when $(\mathbf{pt} + qI + e(\Delta_{C2S}^N))$ is an input of EvalMod or EvalRound step. So this means we can get appropriate output when input is in the range $[\mathbf{pt} + qI - B(\Delta_{C2S}^N), \mathbf{pt} + qI + B(\Delta_{C2S}^N)]$.

In our modified algorithm, we need to get an appropriate output when $(\mathbf{pt} + qI + \Delta_S e_{RS} + e(2\Delta_{C2S}^N))$ is an input of EvalMod or EvalRound step. So we need to get appropriate output when input is in the range $[\mathbf{pt} + qI + \Delta_S e_{RS} - B(\Delta_{C2S}^N)/2, \mathbf{pt} + qI + \Delta_S e_{RS} + B(\Delta_{C2S}^N)/2]$. If $\Delta_S \|e_{RS}\|_\infty \leq B(\Delta_{C2S}^N)/2$ is satisfied, input of EvalMod or EvalRound step for our modified algorithm is in the range $[\mathbf{pt} + qI - B(\Delta_{C2S}^N), \mathbf{pt} + qI + B(\Delta_{C2S}^N)]$. So, in this case, we can guarantee that our algorithm works correctly from the correctness of the baseline algorithm.

3.2.2 Size of Δ_S

Now we estimate the size of Δ_S , which satisfies the condition $\Delta_S \|e_{RS}\|_\infty \leq B(\Delta_{C2S}^N)/2$. To satisfy this condition, it is important to analyze the size of $\|e_{RS}\|_\infty$ and $B(\Delta_{C2S}^N)$. First, the following lemma characterizes the sup-norm of the rescale error in the coefficients, denoted by $\|e_{RS}\|_\infty$.

Lemma 4. *Let e_{RS} be a rescale error in the message on coefficients, and $(1, s)$ be the secret key. And $h \geq 30$ is the Hamming weight of the secret key. And let's assume the ring dimension N to be less than or equal to 2^{18} . Then the following holds.*

$$\Pr \left(\|e_{RS}\|_\infty \leq 16\sqrt{\frac{h+1}{12}} \right) \geq 1 - 2^{-170}.$$

Proof. The rescaling involves division by the rescale factor followed by rounding, introducing an error in $[-\frac{1}{2}, \frac{1}{2}]$ per coefficient. We assume that the rounding error for each coefficient follows a uniform distribution over $[-\frac{1}{2}, \frac{1}{2}]$ [KPK⁺22]. To recover the message coefficients from the ciphertext, we compute the inner product with the secret key $(1, s)$. Interpreting the ciphertext as $(b + e_b, a + e_a)$, where e_b and e_a are rounding errors, the resulting message error is $(e_b, e_a) \cdot (1, s)$. Each message coefficient thus becomes the sum of $h + 1$ independent uniform variables over $[-\frac{1}{2}, \frac{1}{2}]$, with total variance $\frac{h+1}{12}$. When $h > 30$, the distribution of the message error is approximately normal with mean zero.

The probability that a single coefficient exceeds $16\sqrt{\frac{h+1}{12}}$ is $\Pr(|Z| \geq 16) = 1.28 \times 10^{-57}$, where Z is a standard normal variable. Thus, the probability that a single coefficient remains within this bound is $1 - 1.28 \times 10^{-57}$, and so $\Pr \left(\|e_{RS}\|_\infty \leq 16\sqrt{\frac{h+1}{12}} \right) = (1 - 1.28 \times 10^{-57})^N$. Assuming $N \leq 2^{18}$, we have $\Pr \left(\|e_{RS}\|_\infty \leq 16\sqrt{\frac{h+1}{12}} \right) \geq 1 - 1.28 \times 10^{-57} \cdot 2^{18} \geq 1 - 2^{-170}$. $\square$

By the Lemma 4, for sufficiently high probability, $\|e_{RS}\|_\infty \leq 16\sqrt{\frac{h+1}{12}}$ holds.

Now let us consider the term $B(\Delta_{C2S}^N)$. Given the formula $B(\Delta_{C2S}) = \frac{C'}{\Delta_{C2S}}$, it follows that $B(\Delta_{C2S}^N) = B(\Delta_{C2S}^I) \cdot \frac{\Delta_{C2S}^I}{\Delta_{C2S}^N}$. To evaluate $B(\Delta_{C2S}^I)$, which is the error bound of C2S in ordinary CKKS bootstrapping, we consider the final linear transformation in C2S. The rescaling within the last PtMult constitutes the final rescaling operation in C2S, both in the BSGS and naive algorithms. Even if additional operations (excluding RS) follow the last PtMult, the magnitude of the rescale error introduced at that point does not decrease. Let d_{n_c} denote the number of nonzero diagonals in the matrix corresponding to the final linear transformation of C2S. Then, this step involves d_{n_c} PtMult operations. Under an average-case analysis, we can estimate $B(\Delta_{C2S}^I)$ as at least $\sqrt{d_{n_c}}$ times the rescale error. Here, we refer to the rescale error in slots, whose high-probability bound is given by $8\sqrt{Nh/3}$ in [CKKS17].

We have $B(\Delta_{C2S}^I) \geq 8\sqrt{d_{n_c}Nh/3}$, so it follows that $B(\Delta_{C2S}^N) \geq 8\sqrt{d_{n_c}Nh/3} \cdot \frac{\Delta_{C2S}^I}{\Delta_{C2S}^N}$. The condition $\Delta_S \|e_{RS}\|_\infty \leq B(\Delta_{C2S}^N)/2$ then becomes $\Delta_S \|e_{RS}\|_\infty \leq 4\sqrt{d_{n_c}Nh/3} \cdot \frac{\Delta_{C2S}^I}{\Delta_{C2S}^N}$. Using the bound $\|e_{RS}\|_\infty \leq 16\sqrt{(h+1)/12}$, we obtain $\Delta_S \cdot 16\sqrt{(h+1)/12} \leq 4\sqrt{d_{n_c}Nh/3} \cdot \frac{\Delta_{C2S}^I}{\Delta_{C2S}^N}$, and thus the condition for Δ_S becomes approximately

$$\Delta_S \leq \frac{\sqrt{d_{n_c}N}}{2} \cdot \frac{\Delta_{C2S}^I}{\Delta_{C2S}^N}.$$

For Δ_S satisfying this bound, the achievable modulus advantage is approximately $\Delta_S/2^{n_c}$. Substituting the above inequality yields a total modulus advantage of $\frac{\sqrt{d_{n_c}N}}{2^{n_c+1}} \cdot \frac{\Delta_{C2S}^I}{\Delta_{C2S}^N}$.

Table 1: Parameter sets for our modification.

	N	h	q	Δ	Δ_{C2S}^I	Δ_{C2S}^N	n_c	d_{n_c}	Δ_S	$(\Delta_S/2^{n_c})$
A	2^{17}	128	60	50	60	29	4	2^4	40	36
A_0	2^{17}	128	60	50	60	60	4	2^4	9	5
B	2^{16}	192	60	45	58	34	4	2^4	32	28
B_0	2^{16}	192	60	45	58	58	4	2^4	8	4

In Table 1, we calculated the size of the modulus advantage based on parameters provided in the EvalRound paper [KPK⁺22]. The rows A and B are parameters from the EvalRound paper, and they are the case when the baseline algorithm is the EvalRound algorithm. The rows A_0 and B_0 are the corresponding parameters to A and B , which is the case when the baseline algorithm is ordinary CKKS bootstrapping. N is the ring dimension, and h is the hamming weight of the secret key. Other variables were mentioned in the analysis of this section. Except for columns for N and h , the unit for values in the table is bits.

As shown in Table 1, $\log(\Delta_S/2^{n_c})$ corresponds to size of modulus advantage. Our algorithm works with ordinary CKKS bootstrapping, but EvalRound can make the modulus advantage much bigger. As $\log \Delta$ corresponds to one level, we can say that for the cases with EvalRound, we can get around a 2/3 level advantage.

4 Tuple-C2S/S2C with Low Modulus Consumption

In this section, we apply Tuple-CKKS [CKKS23] to reduce the modulus consumption of linear transformations of C2S or S2C, which reduces the modulus consumption of homomorphic multiplications. In the linear transformations of C2S and S2C, we sum many factors that are equal to the number of nonzero diagonals. This can make the *lower part* (Section 2.6) quite large, leading some to think that the modulus advantage of Tuple-CKKS might be smaller in such cases. However, PtMult has smaller errors and slower growth in the lower part compared to standard multiplication. By combining the benefits of PtMult with the drawbacks of summing many terms, we find that the modulus advantage for linear transformations is similar to or better than that of standard multiplication.

Tuple-CKKS [CKKS23] primarily focused on multiplication between two ciphertexts. The authors mentioned that other operations could also be performed in decomposed form; however, they did not provide any analysis or detailed description of these methods. Therefore, in this paper, we present both the techniques and analyses for applying this approach to PtMult and Rot, which we refer to as Tuple-PtMult and Tuple-Rot, respectively. Former definition and analysis of the error and *lower part* growth for Tuple PtMult and Tuple Rotation are covered in Appendix C. Then we can perform a homomorphic linear transformation using Tuple PtMult and Rot, and we will call it as Tuple Linear Transformation. We assume that we perform n_{Lin} linear transformations in a row, and we will not use recombination to obtain the normal ciphertext during the n_{Lin} linear transformations.

For each PtMult without Tuple-CKKS, we perform RS by $\Delta_A \approx q_{\text{div}}q_l$. However, by using Tuple-CKKS, we perform RS by q_l . Thus, for each PtMult in the n_{Lin} linear transformations, we gain a modulus advantage of q_{div} . And we use q_{div} amount of modulus

when we first decompose the ciphertext. Therefore, the modulus advantage we achieve is $q_{\text{div}}^{n_{\text{Lin}}-1}$. To calculate the actual modulus advantage, we need to determine q_{div} and q_l . For that, we will provide an analysis of lower part growth and error analysis for the linear transformation now, and calculate q_{div} and q_l for C2S and S2C in the next section.

Let $d_1, d_2, \dots, d_{n_{\text{Lin}}}$ be the number of nonzero diagonals of each linear transformation matrix. Assume that we are using the Baby-Step Giant-Step algorithm for each linear transformation. In the Equation 2.2 representing Baby-Step Giant-Step linear transformation, $N_1 \cdot N_2 = d_k$ needs to be satisfied for each $1 \leq k \leq n_{\text{Lin}}$, and we set $N_1 \approx N_2 \approx \sqrt{d_k}$ for efficiency. Thus, the k^{th} linear transformation will be performed by applying operations in the following order: Rot, PtMult, summation of $\sqrt{d_k}$ terms, Rot, and summation of $\sqrt{d_k}$ terms.

First, we observe the size of the lower part. Let's think $\check{M}$ as current lower part size, and $\check{M}_{\text{next}}$ as lower part size after some operation. And let $\check{M}_0 = N \cdot q_{\text{div}} \cdot \frac{h+3}{2}$, then by Lemma 1, $\check{M}_0$ represents the bound of $\|\text{can} \circ \text{Dec}(\check{\text{ct}})\|_{\infty}$, where $\check{\text{ct}}$ is the lower part of the decomposed ciphertext that has not undergone any operations. For Tuple Rot, we have following formula by Theorem 4 in Appendix C:

$$\check{M}_{\text{next}} \leq \check{M} + \check{M}_0 + N \cdot E_{ksw}. \quad (4.1)$$

E_{ksw} is Key-Switching error, and $N \cdot E_{ksw}$ is much smaller than $\check{M}_0$. Therefore, when performing Rot, the lower part increases by the addition of $\check{M}_0$.

For Tuple PtMult, we have the formula by Corollary 1 in Appendix C:

$$\check{M}_{\text{next}} \leq \check{M} + \check{M}_0 \cdot \left(\frac{1}{h+3} \cdot \frac{\Delta}{\Delta_A} + 1 \right). \quad (4.2)$$

Here, Δ_A is the scaling factor for the matrix diagonals. We define the constant $C_{\text{PtMult}} = \frac{1}{h+3} \cdot \frac{\Delta}{\Delta_A}$, so that each PtMult increases the lower part by $(C_{\text{PtMult}} + 1) \cdot \check{M}_0$.

For the summation of S terms, the lower part increases by a factor of S in the worst-case scenario. However, throughout this paper, we adopt an average-case analysis, under which the lower part is assumed to grow by a factor of $\sqrt{S}$.

When the initial bound of the lower part is $\check{M}_0$, the first linear transformation, according to Equation 4.1 and 4.2, involves the following growth of the lower part: addition of $\check{M}_0$, addition of $(C_{\text{PtMult}} + 1)\check{M}_0$, multiplication by $d_1^{1/4}$, addition of $\check{M}_0$, and another multiplication by $d_1^{1/4}$. The resulting bound is therefore

$$\left(d_1^{1/2} (C_{\text{PtMult}} + 3) + d_1^{1/4} \right) \check{M}_0,$$

Following the same reasoning, the lower part after n_{Lin} recursive linear transformations is bounded by

$$\left(1 + \frac{1}{d_1^{1/2}} + \dots + \frac{1}{d_1^{1/2} \dots d_{n_{\text{Lin}}-1}^{1/2}} \right) \left(\prod_{i=1}^{n_{\text{Lin}}} d_i^{1/2} \right) (C_{\text{PtMult}} + 4) \check{M}_0.$$

Assuming $d_i \geq 4$ for all i , we further simplify the bound to

$$2\sqrt{d_1 d_2 \dots d_{n_{\text{Lin}}}} (C_{\text{PtMult}} + 4) \check{M}_0.$$

We now analyze the final error after n_{Lin} linear transformations, focusing on the error in the coefficients. Among the sources of error—PtMult, Rot, and summation—PtMult and summation dominate, so we only consider them. Let E_0 denote the base error bound of a PtMult operation on a fresh input. Since the PtMult error scales proportionally with the lower part size, and the lower part after k transformations is bounded by $2\sqrt{d_1 \dots d_k} (C_{\text{PtMult}} + 4) \check{M}_0$, the resulting error bound is approximately $2\sqrt{d_1 \dots d_k} (C_{\text{PtMult}} +$

4) E_0 . After accounting for the summation expansion after the PtMult, the contribution to the final error becomes $2\sqrt{d_1 \cdots d_{n_{\text{Lin}}}}(C_{\text{PtMult}} + 4)E_0$. Summing over all n_{Lin} PtMult, we obtain the final bound:

$$2n_{\text{Lin}}\sqrt{d_1 \cdots d_{n_{\text{Lin}}}}(C_{\text{PtMult}} + 4)E_0. \quad (4.3)$$

4.1 Application to C2S and S2C

C2S and S2C are homomorphic iDFT and DFT operations so that they can be viewed as linear transformations. We estimate the parameters q_{div} and q_l for Tuple-CKKS when applied to these, and evaluate the resulting modulus consumption advantage.

Let E_0 and E_1 be the base coefficient errors for Tuple PtMult and Tuple Mult, respectively. By Theorem 1 and [CKKS23], dominant terms of their errors are:

$$\text{PtMult} : \frac{N\check{M}\|\check{\mathbf{pt}}\|_\infty}{q_{\text{div}}q_l}, \quad \text{Mult} : \frac{N\check{M}^2}{q_{\text{div}}q_l},$$

where $\check{M}$ and $\|\check{\mathbf{pt}}\|_\infty$ are both proportional to q_{div} . Hence, both error terms scale with q_{div}/q_1 . Let h and h_o be the Hamming weights of the secret key for E_0 and E_1 . Then by Lemma 1, for the PtMult, $\check{M} \leq \frac{h+3}{2}q_{\text{div}}$ and $\|\check{\mathbf{pt}}\|_\infty \leq \frac{1}{2}q_{\text{div}}$ holds. And for the multiplication, $\check{M} \leq \frac{h_o+3}{2}q_{\text{div}}$ holds. Taking square root via average case analysis, we obtain:

$$E_0 : E_1 \approx \sqrt{h+3} : (h_o+3). \quad (4.4)$$

Tuple CKKS [CKKS23] used parameters $q_{\text{div}}^o = 2^{23}$, $q_l^o = 2^{38}$, and $h_o = 21845$. Let E_0^o and E_1^o be the corresponding base errors in [CKKS23]. Since error is proportional to q_{div}/q_1 , we have $E_0^o/E_0 = (q_{\text{div}}^o/q_1^o)/(q_{\text{div}}/q_1)$. According to [CKKS23], it is possible to perform 6 consecutive tuple-multiplications without recombination. As each multiplication roughly doubles the lower part size, the error after the final multiplication is about $2^{10}E_1^o$. Using Equation 4.4, it corresponds to $2^{10} \cdot (h_o+3)/\sqrt{h+3} \cdot E_0$.

Our error bound for linear transformations in Equation 4.3 must not exceed the error bound from the result in [CKKS23], so we get the following condition.

$$2^{10} \cdot (h_o+3)/\sqrt{h+3} \cdot E_0 = 2n_{\text{Lin}}\sqrt{d_1 \cdots d_{n_{\text{Lin}}}}(C_{\text{PtMult}} + 4)E_0.$$

Using $E_0^o/E_0 = (q_{\text{div}}^o/q_1^o)/(q_{\text{div}}/q_1)$, we finally derive:

$$\frac{q_{\text{div}}}{q_1} = \frac{2^{10}(h_o+3)/\sqrt{h+3} \cdot (q_{\text{div}}^o/q_1^o)}{2n_{\text{Lin}}\sqrt{d_1 \cdots d_{n_{\text{Lin}}}}(C_{\text{PtMult}} + 4)}.$$

Given the constraint $q_{\text{div}}q_1 = \Delta_{\text{C2S}}$ or $q_{\text{div}}q_1 = \Delta_{\text{S2C}}$, we can solve for both q_{div} and q_l .

Finally, we apply this computation to each parameter set from Lattigo [MBTPH20], where N is the ring dimension, h is the Hamming weight, and log-scales of q and Δ are given. For simplicity, we assume $C_{\text{PtMult}} \leq 1$, which holds for all Lattigo parameters. The resulting modulus advantage is compared to that of ordinary CKKS bootstrapping. The size of Δ corresponds to one level, so from the Table 2, we observe that there is an advantage of more than two levels, except for the lowest precision case IV.

5 Putting It All Together

This section considers the case where all three methods—OverModRaise1, OverModRaise2, and Tuple Linear Transformation—are used together. Let n_c and n_s denote the number

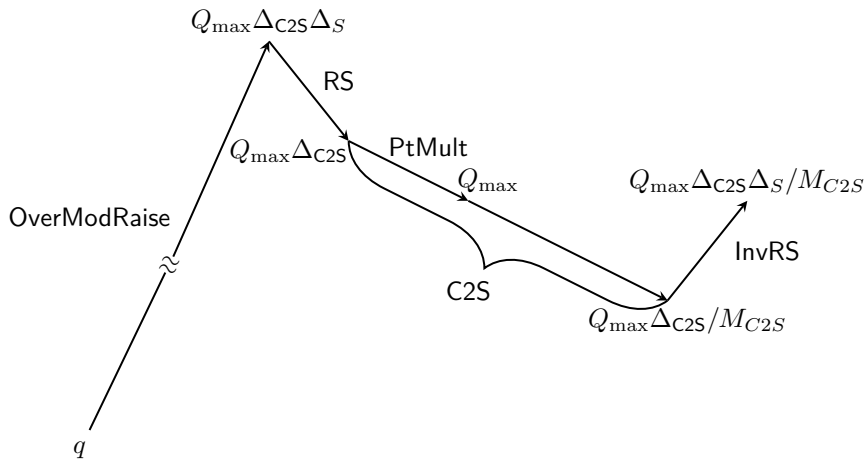
Table 2: Modulus advantages of Tuple-linear transformation (Tuple-C2S/S2C) compared to ordinary CKKS bootstrapping based on Lattigo parameter sets (I-V).

	N	h	q	Δ	C2S	q_{div}^C	S2C	q_{div}^S	adv.O
I	2^{16}	192	60	40	56×4	23×1	39×3	15×1	99
II	2^{16}	192	60	45	58×4	24×1	42×3	16×1	104
III	2^{16}	192	55	30	53×4	22×1	30×3	10×1	86
IV	2^{15}	192	32	25	49×2	20×1	30×2	11×1	31
V	2^{16}	32768	60	45	58×4	22×1	42×3	14×1	94

of decomposed matrices in C2S and S2C, respectively. Let q_{div}^C and q_l^C denote the q_{div} and q_l values for C2S, and q_{div}^S and q_l^S those for S2C. **OverModRaise1** and **OverModRaise2** only modify the C2S component, while Tuple Linear Transformation can be applied to both C2S and S2C. Therefore, for S2C, we simply adopt the Tuple Linear Transformation. Consequently, the modulus advantage for S2C will be $(q_{\text{div}}^S)^{n_s-1}$.

Remark 1 (C2S with Proposed Techniques). While all three methods can be applied to C2S, adding the tuple linear transformation on top of **OverModRaise1** and 2 yields no further modulus savings. This is because **OverModRaise1** and 2 already reduce the relevant parameter in C2S, limiting the additional benefit of applying the Tuple Linear Transformation.

In **OverModRaise1**, we **ModRaise** into $Q_{\text{max}}\Delta_{\text{C2S}}$, which is Δ_{C2S} times larger than Q_{max} . In **OverModRaise2**, we **ModRaise** into $Q_{\text{max}}\Delta_S$, which is Δ_S times larger than Q_{max} . To combine these, we **ModRaise** into the modulus $Q_{\text{max}}\Delta_{\text{C2S}}\Delta_S$. After **ModRaise**, we perform **RS** by Δ_S , resulting in a modulus of $Q_{\text{max}}\Delta_{\text{C2S}}$. From this point, we can apply the same method as in Section 3.1, which involves switching the order of **Rot** and **PtMult** in the first linear transformation of C2S. After C2S, we perform **InvRS** by Δ_S . Here, the modulus advantage Δ_{C2S} from **OverModRaise1** and the modulus advantage $\Delta_S/2^{n_c}$ from **OverModRaise2** are combined. In conclusion, by integrating **OverModRaise1** and 2, we achieve a modulus advantage of $\Delta_{\text{C2S}}\Delta_S/2^{n_c}$ for C2S. Figure 3 and Algorithm 1 illustrate the modulus transitions and the corresponding procedure until C2S evaluation, respectively. After the steps shown in the figure, **EvalRound** will be applied.

**Figure 3:** Modulus change of our modification

Finally, by combining the modulus advantages from C2S and S2C, we obtain a total

Algorithm 1 CKKS Bootstrapping with OverModRaise1 and OverModRaise2**Input:** ct_{in} Let $A_1, \dots, A_{n_c}$ be the decomposition of iDFT matrix for C2S, and this should also be done to the DFT matrix for S2C.**Step 0:** Preprocessing

for $j = 0, \dots, N/2 - 1$ do
 $\text{pt}_j \leftarrow \text{Ecd}(\rho(u_j; -j), \Delta_A)$

Step 1: OverModRaise

$\text{ct}_{\text{OverMR}} \leftarrow \text{OverModRaise}(\text{ct}_{\text{in}})$ $\triangleright$ Modulus raising to $Q_{\max} \Delta_{\text{C2S}} \Delta_S$ beyond the maximum ciphertext modulus $Q_{\max}$ (Section 3)

Step 2: Modified C2S**Step 2-1:** Modified the first matrix multiplication of C2S

$\text{ct}_{\text{DS}} \leftarrow \text{RS}_{\Delta_S}(\text{ct}_{\text{OverMR}})$ $\triangleright$ Ciphertext downscaling (Section 3.2)
 $\text{ct}_{A_1} \leftarrow \text{PtMult}(\text{pt}_0, \text{ct}_{\text{DS}})$ $\triangleright$ Modified matrix multiplication (Section 3.1)
 for $j = 1, \dots, N/2 - 1$ do
 $\text{ct}_{\text{temp},j} \leftarrow \text{PtMult}(\text{pt}_j, \text{ct}_{\text{DS}})$
 $\text{ct}_{A_1} \leftarrow \text{ct}_{A_1} + \text{Rot}_j(\text{ct}_{\text{temp},j})$

Step 2-2: Original rest of C2S $\triangleright$ The sequential matrix evaluation of $A_2, \dots, A_{n_c}$ on ct_{A_1} , which is the same as the original CKKS C2S from the second matrix, and output $\text{ct}_{A_{n_c}}$

for $i = 2, \dots, n_c$ do
 $\text{ct}_{A_i} \leftarrow \text{CKKS-BTS.C2S.MatMul}(A_i, \text{ct}_{A_{i-1}})$

Step 2-3: Inverse-Rescaling $\triangleright$ (Section 3.2) $\text{ct}_{\text{C2S}} \leftarrow \text{InvRS}(\text{ct}_{A_{n_c}})$ **Step 3:** Original rest of CKKS bootstrapping procedures

$\text{ct}_{\text{EvalMod}} \leftarrow \text{CKKS-BTS.EvalMod}(\text{ct}_{\text{C2S}})$
 $\text{ct}_{\text{out}} \leftarrow \text{CKKS-BTS.S2C}(\text{ct}_{\text{EvalMod}})$

return ct_{out}

modulus advantage of $\Delta_S \Delta_{\text{C2S}} (q_{\text{div}}^S)^{n_s-1} / 2^{n_c}$ compared to the EvalRound algorithm. Now, we use the formula $\Delta_S \leq \sqrt{d_{n_c} N} / 2^{\frac{\Delta_{\text{C2S}}^I}{\Delta_{\text{C2S}}^E}}$ from Section 3.2. As in Section 3.2, we will set $\Delta_{\text{C2S}} = 2 \Delta_{\text{C2S}}^E$. Consequently, the modulus advantage becomes $\sqrt{d_{n_c} N} \Delta_{\text{C2S}}^I \cdot (q_{\text{div}}^S)^{n_s-1} / 2^{n_c}$ compared to the EvalRound algorithm. This indicates that our modulus advantage, compared to the EvalRound algorithm, is independent of Δ_{C2S}^E .

Remark 2 (Advantages of OverModRaise and Tuple-S2C). The above advantage ($\sqrt{d_{n_c} N} \Delta_{\text{C2S}}^I / 2^{n_c} \cdot (q_{\text{div}}^S)^{n_s-1}$) consists of two parts: 1) $\sqrt{d_{n_c} N} \Delta_{\text{C2S}}^I / 2^{n_c}$ from OverModRaise1 and 2 and 2) $(q_{\text{div}}^S)^{n_s-1}$ from Tuple-S2C. We note that the advantage 1) is close to the size of the base prime q .

In Table 3 we include the Lattigo parameters [MBTPH20] and extend the table with additional parameters and their corresponding advantages. For the parameter sets in Table 3, we set $\Delta_{\text{C2S}}^E = \Delta_{\text{C2S}}^I / 2^{21}$, which is a safe and conservative choice. In Table 3, N represents the ring dimension, and h denotes the Hamming weight of the secret key. q is the base modulus at the lowest level, and Δ is the scaling factor of the ciphertext. Δ_S is

Table 3: Modulus advantages of using all three methods compared to ordinary CKKS bootstrapping based on Lattigo parameter sets (I-V) [MBTPH20].

	N	h	q	Δ	Δ_S	C2S	S2C	q_{div}^S	adv.O
I	2^{16}	192	60	40	30	36×4	56×3	23×1	141
II	2^{16}	192	60	45	30	38×4	58×3	24×1	148
III	2^{16}	192	55	30	30	33×4	53×3	22×1	118
IV	2^{15}	192	32	25	23	28×2	49×2	20×1	75
V	2^{16}	32768	60	45	30	38×4	58×3	22×1	144

the parameter for OverModRaise2. For the C2S and S2C columns, we list the scaling factor and the number of linear transformations. In the q_{div}^S and q_1^S columns, we present the calculated parameters for Tuple-CKKS on S2C. The adv.O column provides the modulus advantage compared to the ordinary bootstrapping. Except for the columns for N and h , the units for the values in the table are in bits. The size of Δ corresponds to one level, so there is more than a 3-level advantage, except for the lowest precision case, IV.

6 Experiments

In this section, we present the experimental results for our proposed methods. We conducted a proof-of-concept implementation³ using the C++ code from [KPK⁺22, Cry24].

Environment. All the experiments are performed in Intel(R) Xeon(R) Silver 4114 CPU @ 2.20GHz with a single thread. We implemented and combined all three proposed methods, evaluating their performance against ordinary bootstrapping and the EvalRound algorithm from [KPK⁺22]. Our experiments include cases using each method individually and all together.

Baseline Implementation and Optimizations. As the state-of-the-art, we consider EvalRound+, adding only skip connections to the public EvalRound implementation [KPK⁺22, Cry24], and the EvalRound implementation does not include BSGS [HS18], hybrid key-switching [HK20], double-hoisting [BMTH21], Sparse Secret Encapsulation (SSE) [BTH22], or lazy BSGS [LLK⁺22].

We note that double-hoisting [BMTH21] cannot be applied to the first linear transformation in C2S when using OverModRaise1. On the other hand, SSE [BTH22] can be used together with our method, as it involves switching to a sparse key, performing OverModRaise, carrying out PtMult or RS, switching back to the original key, and then executing any subsequent operations that require key switching.

6.1 C2S-First Bootstrapping

Parameters. All parameter sets are based on Lattigo’s ‘I’ set [MBTPH20] (Table 3), as detailed in Sections 4 and 5. The parameter is based on the ring dimension $N = 2^{16}$, the secret key Hamming weight $h = 192$, 40 bits of the ciphertext scaling factor Δ , 1546 bits of PQ_{max} , and the discrete Gaussian error distribution with the standard deviation 3.2, which has a security $\lambda \approx 128$ [BTH22]. However, for simplicity, all the experimental results and our implementation—built upon EvalRound—use a simplified error distribution⁴ with the standard deviation of approximately 4.7 to reproduce only the performance corresponding

³Our C++ implementation code is publicly available at a GitHub repository: <https://github.com/aqua4689jh/OverModRaise>.

⁴A discrete random distribution with probability 1/9 at 0 and 1/18 at each of $\pm 1, \dots, \pm 8$.

to the Lattigo parameter sets. The prime sizes allocated for the available multiplications after bootstrapping are $\{60, 40 \times 9\}$ bits in the ordinary case. The special modulus P for key-switching consists of $\{61 \times 5\}$ bits, while for EvalMod, the allocation is $\{60 \times 8\}$ bits.

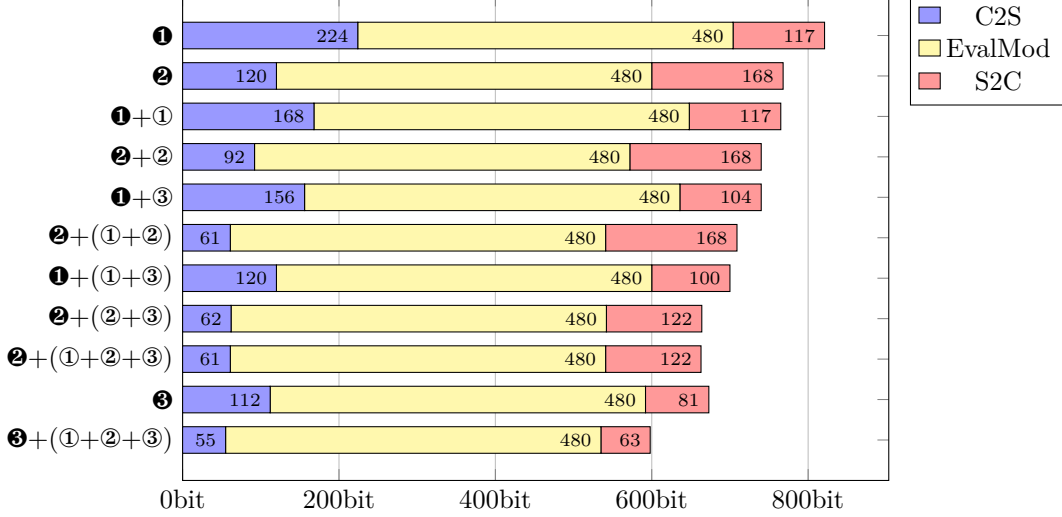


Figure 4: Experimental modulus consumption breakdown results between the proposed techniques (① OverModRaise1, ② OverModRaise2, ③ Tuple-C2S/S2C) over the prior works (① Ordinary CKKS Bootstrapping, ② EvalRound [KPK⁺22], and ③ EvalRound+ [SSKM25]). The numbers represent the modulus bits consumed in each step of the bootstrapping procedure. The extra modulus gained from OverModRaise is represented as a reduction in the modulus consumption of C2S.

Results. Figure 4 summarizes our experimental results between the proposed techniques (① OverModRaise1, ② OverModRaise2, and ③ Tuple-C2S/S2C) over the prior works (① Ordinary CKKS Bootstrapping, ② EvalRound [KPK⁺22], and ③ EvalRound+ [SSKM25]). The numbers on each bar indicate the modulus bits consumed at the corresponding step, while the gains from OverModRaise1 and OverModRaise2 are represented as the reduction in the modulus consumption of C2S. Since EvalMod follows the same procedure as in the baseline, it shows the same modulus consumption. The details of the prime sizes and parameters can be found in Table 5 in Appendix A.

The ③+(①+②+③) case, combining our three methods with the EvalRound+ algorithm, achieved a 75-bit advantage with nearly identical precision. Given a 40-bit scaling factor, this corresponds to roughly two levels. As shown in Table 5 in Appendix A, comparing ② and ②+(①+②), we observe a modulus advantage of 59, corresponding to a 60-bit base prime q , as analyzed in Section 5 (see Remark 2).

Some parameters, such as q_{div}^S , are smaller than $2N$, making them incompatible with typical NTT primes used in RNS-CKKS [CHK⁺19]. This can be resolved using the grafting technique [CCK⁺24], which allows rescaling with smaller-than-NTT moduli. Hence, our method can be implemented in RNS-CKKS using Grafting [CCK⁺24].

6.2 S2C-First Bootstrapping

Parameters. In Table 4, the results for $N = 2^{16}$ are based on Lattigo’s II set [MBTPH20], while those for $N = 2^{15}$ are based on the IV set. We used the same error distribution as in Section 6.1. Detailed parameter settings can be found in Table 6 in the Appendix B.

Table 4: Experimental results of proposed techniques (① OverModRaise1, ② OverModRaise2, ③ Tuple-C2S/S2C) over EvalRound+ [SSKM25].

N	Bootstrapping	C2S	S2C	q_{div}^S	Δ_S	Mult level	Precision	Latency	Throughput
2^{16}	EvalRound+	37	42	–	–	6	33.46	$1\times$	$1\times$
	with ①+②	38	42	–	30	8	33.48	$1.07\times$	$1.247\times$
	with ②+③	38	42	16	30	8	33.64	$1.10\times$	$1.218\times$
	with ①+②+③	38	42	16	30	9	33.60	$1.14\times$	$1.318\times$
2^{15}	EvalRound+	27	30	–	–	4	28.90	$1\times$	$1\times$
	with ①+②	28	30	–	23	6	28.90	$1.29\times$	$1.163\times$
	with ②+③	28	30	10	23	6	29.04	$1.07\times$	$1.406\times$
	with ①+②+③	28	30	10	23	7	29.04	$1.34\times$	$1.316\times$

Results. Table 4 presents the relative runtime overhead (Latency column) and relative throughput improvement (Throughput column) in CKKS bootstrapping. The throughputs are derived by the formula $(\text{Throughput}) = (\# \text{ of mult levels}) \times (\text{Precision}) / (\text{Latency})$ from [CHKS25]. With the three proposed methods, we achieve a **40.6% increase in throughput** compared to the state-of-the-art. For $N = 2^{16}$, the original multiplication depth is larger, so the benefit of additional levels is smaller. Hence, using only OverModRaise1 and 2 gives better results.

The runtime overhead from OverModRaise2 is negligible, since the added RS and InvRS operations are even lighter than a single ciphertext-ciphertext multiplication (ctMult). With proper decomposition of the C2S and S2C matrices (see Section 3.1), OverModRaise1 also introduces minimal overhead. As shown in Table 4, the actual overhead from OverModRaise1 and 2 is indeed negligible in practice.

Applying Tuple-CKKS to S2C remains efficient when using the S2C-first bootstrapping of [BCC⁺22], since S2C is computed on a small modulus. Although the cost of S2C doubles with Tuple-CKKS, the overall increase is small, as can be seen in Table 4. Note that S2C-first bootstrapping is compatible with EvalRound+ but not with EvalRound. While our discussion is based on C2S-first bootstrapping, all three methods are equally applicable to S2C-first if the base algorithm is either standard or EvalRound+.

7 Application and Impact

OverModRaise and Tuple-S2C/C2S exploit the latency–modulus trade-off to improve throughput, which directly reduces the required number of bootstrapping operations and thus the overall runtime in practice. For ResNet-110 [CKK⁺25], EvalRound+ requires 327 bootstraps, whereas our scheme requires only 218 for Lattigo’s II set of the ring dimension $N = 2^{16}$. This skips about 109 modulus cycles (from Q_{max} down to the base prime) and induces a $1.14\times$ longer runtime per bootstrapping. As a result, the latency can be reduced to 76%, which is close to the inverse throughput value of $1/1.318$.

Acknowledgments

Jung Hee Cheon and Yongdong Yeo were financially supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korean government (MSIT) (RS-2025-02217656). We thank Hyeongmin Choe and Minsik Kang for their helpful discussions that improved this work. We are also grateful to

the reviewers, the editor, and the organizing committee of Communications in Cryptology 2025 for their valuable feedback and support.

References

- [BCC⁺22] Youngjin Bae, Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Taekyung Kim. META-BTS: Bootstrapping precision beyond the limit. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 223–234. ACM Press, November 2022. doi:[10.1145/3548606.3560696](https://doi.org/10.1145/3548606.3560696).
- [BMTH21] Jean-Philippe Bossuat, Christian Mouchet, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 587–617. Springer, Cham, October 2021. doi:[10.1007/978-3-030-77870-5_21](https://doi.org/10.1007/978-3-030-77870-5_21).
- [BTH22] Jean-Philippe Bossuat, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. Bootstrapping for approximate homomorphic encryption with negligible failure-probability by using sparse-secret encapsulation. In Giuseppe Ateniese and Daniele Venturi, editors, *ACNS 22International Conference on Applied Cryptography and Network Security*, volume 13269 of *LNCS*, pages 521–541. Springer, Cham, June 2022. doi:[10.1007/978-3-031-09234-3_26](https://doi.org/10.1007/978-3-031-09234-3_26).
- [CCK⁺24] Jung Hee Cheon, Hyeongmin Choe, Minsik Kang, Jaehyung Kim, Seonghak Kim, Johannes Mono, and Taeyeon Noh. Grafting: Decoupled scale factors and modulus in RNS-CKKS. Cryptology ePrint Archive, Paper 2024/1014, 2024. URL: <https://eprint.iacr.org/2024/1014>.
- [CCKS23] Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Damien Stehlé. Homomorphic multiple precision multiplication for CKKS and reduced modulus consumption. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 696–710. ACM Press, November 2023. doi:[10.1145/3576915.3623086](https://doi.org/10.1145/3576915.3623086).
- [CCS19] Hao Chen, Ilaria Chillotti, and Yongsoo Song. Improved bootstrapping for approximate homomorphic encryption. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part II*, volume 11477 of *LNCS*, pages 34–54. Springer, Cham, May 2019. doi:[10.1007/978-3-030-17656-3_2](https://doi.org/10.1007/978-3-030-17656-3_2).
- [CHK⁺18] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part I*, volume 10820 of *LNCS*, pages 360–384. Springer, Cham, April / May 2018. doi:[10.1007/978-3-319-78381-9_14](https://doi.org/10.1007/978-3-319-78381-9_14).
- [CHK⁺19] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. A full RNS variant of approximate homomorphic encryption. In Carlos Cid and Michael J.: Jacobson, Jr., editors, *SAC 2018*, volume 11349 of *LNCS*, pages 347–368. Springer, Cham, August 2019. doi:[10.1007/978-3-030-10970-7_16](https://doi.org/10.1007/978-3-030-10970-7_16).
- [CHKS25] Jung Hee Cheon, Guillaume Hanrot, Jongmin Kim, and Damien Stehlé. SHIP: A shallow and highly parallelizable CKKS bootstrapping algorithm.

- In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025, Part III*, volume 15603 of *Lecture Notes in Computer Science*, pages 398–428, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland. doi:10.1007/978-3-031-91131-6_14.
- [CKK⁺25] Jung Hee Cheon, Minsik Kang, Taeseong Kim, Junyoung Jung, and Yongdong Yeo. Batch inference on deep convolutional neural networks with fully homomorphic encryption using channel-by-channel convolutions. *IEEE Transactions on Dependable and Secure Computing*, 22(2):1674–1685, 2025. doi:10.1109/TDSC.2024.3448406.
- [CKKS17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. Homomorphic encryption for arithmetic of approximate numbers. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part I*, volume 10624 of *LNCS*, pages 409–437. Springer, Cham, December 2017. doi:10.1007/978-3-319-70694-8_15.
- [Cry24] CryptoLabInc. Evalround: Github repository, 2024. Accessed: 2024-10-23. URL: <https://github.com/CryptoLabInc/EvalRound>.
- [HHC19] Kyoohyung Han, Minki Hhan, and Jung Hee Cheon. Improved homomorphic discrete fourier transforms and the bootstrapping. *IEEE Access*, 7:57361–57370, 2019. doi:10.1109/ACCESS.2019.2913850.
- [HK20] Kyoohyung Han and Dohyeong Ki. Better bootstrapping for approximate homomorphic encryption. In Stanislaw Jarecki, editor, *CT-RSA 2020*, volume 12006 of *LNCS*, pages 364–390. Springer, Cham, February 2020. doi:10.1007/978-3-030-40186-3_16.
- [HS14] Shai Halevi and Victor Shoup. Algorithms in HELib. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 554–571. Springer, Berlin, Heidelberg, August 2014. doi:10.1007/978-3-662-44371-2_31.
- [HS18] Shai Halevi and Victor Shoup. Faster homomorphic linear transformations in HELib. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 93–120. Springer, Cham, August 2018. doi:10.1007/978-3-319-96884-1_4.
- [HS21] Shai Halevi and Victor Shoup. Bootstrapping for HELib. *Journal of Cryptology*, 34(1):7, January 2021. doi:10.1007/s00145-020-09368-7.
- [JM20] Charanjit S. Jutla and Nathan Manohar. Modular lagrange interpolation of the mod function for bootstrapping for approximate HE. Cryptology ePrint Archive, Report 2020/1355, 2020. URL: <https://eprint.iacr.org/2020/1355>.
- [KLK⁺22] Jongmin Kim, Gwangho Lee, Sangpyo Kim, Gina Sohn, Minsoo Rhu, John Kim, and Jung Ho Ahn. Ark: Fully homomorphic encryption accelerator with runtime data generation and inter-operation key reuse. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1237–1254, 2022. doi:10.1109/MICRO56248.2022.00086.
- [KPK⁺22] Seonghak Kim, Minji Park, Jaehyung Kim, Taekyung Kim, and Chohong Min. EvalRound algorithm in CKKS bootstrapping. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part II*, volume 13792 of *LNCS*, pages 161–187. Springer, Cham, December 2022. doi:10.1007/978-3-031-22966-4_6.

- [LLK⁺22] Yongwoo Lee, Joon-Woo Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and HyungChul Kang. High-precision bootstrapping for approximate homomorphic encryption by error variance minimization. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 551–580. Springer, Cham, May / June 2022. doi:10.1007/978-3-031-06944-4_19.
- [LLL⁺21] Joon-Woo Lee, Eunsang Lee, Yongwoo Lee, Young-Sik Kim, and Jong-Seon No. High-precision bootstrapping of RNS-CKKS homomorphic encryption using optimal minimax polynomial approximation and inverse sine function. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 618–647. Springer, Cham, October 2021. doi:10.1007/978-3-030-77870-5_22.
- [MBTPH20] Christian Vincent Mouchet, Jean-Philippe Bossuat, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. Lattigo: A multiparty homomorphic encryption library in go. In *Proceedings of the 8th Workshop on Encrypted Computing and Applied Homomorphic Cryptography*, pages 64–70, 2020.
- [SSKM25] Hyewon Sung, Sieun Seo, Taekyung Kim, and Chohong Min. Evalround+ bootstrapping and its rigorous analysis for ckks scheme. *IEEE Access*, 13:140847–140866, 2025. doi:10.1109/ACCESS.2025.3595177.

A Experimental Results for Section 6.1

Table 5: Experimental results between the proposed techniques (① OverModRaise1, ② OverModRaise2, ③ Tuple-C2S/S2C) over the prior works (❶ Ordinary CKKS Bootstrapping, ❷ EvalRound [KPK⁺22], and ❸ EvalRound+ [SSKM25]).

Bootstrapping	C2S	S2C	Δ_S	q_{div}^C	q_{div}^S	mod.	vs.❶	vs.❸	precision
❶	56	39	-	-	-	821	-	-	26.30
❷	30	56	-	-	-	768	53	-	27.09
❶+①	56	39	-	-	-	765	56	-	26.86
❷+②	31	56	32	-	-	740	81	-	28.51
❶+③	60	50	-	28×1	23×1	740	81	-	27.24
❷+(①+②)	31	56	32	-	-	709	112	-	27.35
❶+(①+③)	60	50	-	30×1	25×1	700	121	-	26.39
❷+(②+③)	31	56	32	10×1	23×1	664	157	9	26.67
❷+(①+②+③)	31	56	32	-	23×1	663	158	10	26.52
❸	28	27	-	-	-	673	148	-	26.46
❸+(①+②+③)	29	27	32	-	9×1	598	223	75	26.08

The prime sizes for C2S and S2C vary across different setups and are listed in the table, while their counts remain fixed at 4 and 3, respectively. The Δ_S column indicates the bit size used in the Rescale step (Section 3.2), which replaces EvalMod with EvalRound. The parameters ' q_{div}^C ', and ' q_{div}^S ' are Tuple-CKKS primes used in 'C2S' and 'S2C'. Result columns include: 'mod.' (total modulus consumption in bits), 'vs.❶' and 'vs.❸' (reduced modulus consumption in bits over ordinary and EvalRound+ [SSKM25]), and 'precision', computed as $-\log_2 \|pt/\Delta - \hat{pt}/\Delta\|_\infty$. Blank cells indicate parameters that are unused or not meaningful in the corresponding place.

B Parameter Sets and Additional Results for Section 6.2

Table 6 presents the detailed parameter settings for Lattigo’s II and IV sets [MBTPH20], which are used in Section 6.2. In Table 6, N denotes the ring dimension and h the Hamming weight of the secret key. q is the base modulus at the lowest level. The Δ column lists the scaling factor used for multiplication and the number of multiplication levels. $PQ_{\max}$ is the maximum modulus, and the P column specifies the size and number of primes used for Key-Switching. The C2S and S2C columns provide the scaling factor and the number of linear transformations, respectively. The EvalMod column contains the size and number of primes used for EvalMod. Except for N and h , all values are given in bits. For the number of multiplication levels and the scaling factor for C2S, Table 6 shows the original Lattigo parameters [MBTPH20], while Table 4 presents our modified values.

Table 6: Parameter sets for using all three methods.

	N	h	q	Δ	$PQ_{\max}$	P	C2S	S2C	EvalMod
II	2^{16}	192	60	45×5	1547	61×5	58×4	42×3	60×11
IV	2^{15}	192	32	25×3	768	51×2	49×2	30×2	50×8

Table 7 presents the relative runtime overhead (Latency column) and relative throughput improvement (Throughput column) achieved when applying our three methods to ordinary CKKS bootstrapping.

Table 7: Experimental results of proposed techniques (① OverModRaise1, ② OverModRaise2, ③ Tuple-C2S/S2C) over Ordinary CKKS Bootstrapping.

N	Bootstrapping	C2S	S2C	q_{div}^S	Δ_S	Mult level	Precision	Latency	Throughput
2^{16}	Ordinary	58	42	—	—	5	33.47	$1 \times$	$1 \times$
	with ①+②	59	42	—	8	6	31.65	$1.05 \times$	$1.080 \times$
	with ②+③	59	42	16	8	5	34.64	$1.14 \times$	$0.910 \times$
	with ①+②+③	59	42	16	8	7	32.25	$1.16 \times$	$1.161 \times$
2^{15}	Ordinary	49	30	—	—	3	28.90	$1 \times$	$1 \times$
	with ①+②	50	30	—	8	5	29.59	$1.46 \times$	$1.170 \times$
	with ②+③	50	30	10	8	3	29.56	$1.10 \times$	$0.926 \times$
	with ①+②+③	50	30	10	8	5	29.37	$1.54 \times$	$1.098 \times$

C Tuple-CKKS for PtMult and Rot

This section provides formal definitions and analysis of the error and *lower part* growth for Tuple PtMult and Tuple Rotation.

C.1 Tuple-CKKS on PtMult

Definition 5 (Tensor Product Between Plaintext and Ciphertext). Let $\text{pt} \in R_{Q_l}$ and $\text{ct} = (b, a) \in R_{Q_l}^2$ represent a plaintext and a ciphertext, respectively. The tensor product between the plaintext and ciphertext is defined as

$$\text{pt} \otimes \text{ct} = (\text{pt} \cdot b, \text{pt} \cdot a)$$

Definition 6 (Tuple Tensor). Let $\text{PT} = (\hat{\text{pt}}, \check{\text{pt}}) \in R_{Q_l} \times R_{Q_l}$ and $\text{CT} = (\hat{\text{ct}}, \check{\text{ct}}) \in R_{Q_l}^2 \times R_{Q_l}^2$ represent decomposed plaintext and ciphertext, respectively. The $\otimes^2$ operation between PT and CT is defined as

$$\text{PT} \otimes^2 \text{CT} = (\hat{\text{pt}} \otimes \hat{\text{ct}}, \hat{\text{pt}} \otimes \check{\text{ct}} + \check{\text{pt}} \otimes \hat{\text{ct}}) \in R_{Q_l}^2 \times R_{Q_l}^2$$

We will also refer to the above as $\text{Tensor}^2(\text{PT}, \text{CT})$.

The formula for Tensor^2 between plaintext and ciphertext is identical to the Tensor^2 between ciphertext and ciphertext, except for the spaces of the elements.

Lemma 5 (Error of Tuple Tensor). Let $\text{PT} = (\hat{\text{pt}}, \check{\text{pt}}) \in R_{Q_l} \times R_{Q_l}$ and $\text{CT} = (\hat{\text{ct}}, \check{\text{ct}}) \in R_{Q_l}^2 \times R_{Q_l}^2$ represent decomposed plaintext and ciphertext, respectively, with $\text{RCB}_{q_{\text{div}}}(\text{PT}) = \text{pt}$ and $\text{RCB}_{q_{\text{div}}}(\text{CT}) = \text{ct}$. Let $\text{sk} = (1, s) \in R^2$ denote the secret key. Then, the following formula is satisfied.

$$(\text{pt} \otimes \text{ct}) \cdot (1, s) = q_{\text{div}} \cdot (\text{RCB}_{q_{\text{div}}}(\text{PT} \otimes^2 \text{CT})) \cdot (1, s) + \check{\text{pt}} \cdot (\check{\text{ct}} \cdot \text{sk}) \pmod{Q_l}.$$

Let's assume $\|\text{Dec}(\hat{\text{ct}})\| \leq \hat{M}$ and $\|\text{Dec}(\check{\text{ct}})\| \leq \check{M}$. Also assume that $N\|\text{pt}\|_{\infty}(\hat{M}q_{\text{div}} + \check{M}) < Q_l/2$. Then the inequality below is satisfied.

$$\|[\text{RCB}_{q_{\text{div}}}(\hat{T}, \check{T}) \cdot (1, s)]_{Q_l} - \frac{1}{q_{\text{div}}}[(\text{pt} \otimes \text{ct}) \cdot (1, s)]_{Q_l}\|_{\infty} \leq \frac{N\check{M}\|\check{\text{pt}}\|_{\infty}}{q_{\text{div}}}.$$

Proof. Let $\hat{m} = \text{Dec}_{\text{sk}}(\hat{\text{ct}})$ and $\check{m} = \text{Dec}_{\text{sk}}(\check{\text{ct}})$. From the definition above and the definition of recombination, we have,

$$\text{RCB}_{q_{\text{div}}}(\text{PT} \otimes^2 \text{CT}) = q_{\text{div}} \cdot (\hat{\text{pt}} \otimes \hat{\text{ct}}) + (\hat{\text{pt}} \otimes \check{\text{ct}} + \check{\text{pt}} \otimes \hat{\text{ct}}).$$

Now in modulo Q_l ,

$$(\text{RCB}_{q_{\text{div}}}(\text{PT} \otimes^2 \text{CT})) \cdot (1, s) = q_{\text{div}} \cdot (\hat{\text{pt}} \hat{m}) + (\hat{\text{pt}} \check{m} + \check{\text{pt}} \hat{m}).$$

Also, the below holds in modulo Q_l .

$$\begin{aligned} (\text{pt} \otimes \text{ct}) \cdot (1, s) &= ((q_{\text{div}} \cdot \hat{\text{pt}} + \check{\text{pt}}) \otimes (q_{\text{div}} \cdot \hat{\text{ct}} + \check{\text{ct}})) \cdot (1, s) \\ &= q_{\text{div}}^2(\hat{\text{pt}} \hat{m}) + q_{\text{div}}(\hat{\text{pt}} \check{m} + \check{\text{pt}} \hat{m}) + (\check{\text{pt}} \check{m}). \end{aligned}$$

By combining the two formulas above, we obtain the first result. Given the condition $N\|\text{pt}\|_{\infty}(\hat{M}q_{\text{div}} + \check{M}) < Q_l/2$, both the left-hand side and right-hand side of the first result have infinity norms less than $Q_l/2$. Therefore, the following holds in the space R .

$$[(\text{pt} \otimes \text{ct}) \cdot (1, s)]_{Q_l} = q_{\text{div}} \cdot [(\text{RCB}_{q_{\text{div}}}(\text{PT} \otimes^2 \text{CT})) \cdot (1, s)]_{Q_l} + [\check{\text{pt}}]_{Q_l} \cdot [\check{\text{ct}} \cdot \text{sk}]_{Q_l}.$$

By bounding the $[\check{\text{pt}}]_{Q_l} \cdot [\check{\text{ct}} \cdot \text{sk}]_{Q_l}$ term, we get the second result. $\square$

We have defined Tuple Tensor for Tuple PtMult. Notably, the Relinearization step is not required for PtMult. For the Rescale step in PtMult, we can utilize the Tuple Rescale introduced in the Tuple-CKKS paper [CKKS23]. With these definitions, we can now formally define Tuple PtMult.

Definition 7 (Tuple PtMult). We define PtMult between decomposed plaintext and ciphertext as $\text{PtMult}^2 = \text{RS}_{q_l}^2 \circ \text{Tensor}^2$, where $q_l = Q_l/Q_{l-1}$. The output is a decomposed ciphertext with modulus Q_{l-1} .

Using Lemma 5 and the error analysis for Tuple Rescale in the Tuple-CKKS paper [CKKS23], we can derive the following theorem.

Theorem 1 (Error of Tuple PtMult). *Let $\text{PT} = (\hat{\text{pt}}, \check{\text{pt}}) \in R_{Q_l} \times R_{Q_l}$, $\text{CT} = (\hat{\text{ct}}, \check{\text{ct}}) \in R_{Q_l}^2 \times R_{Q_l}^2$ be decomposed plaintext and ciphertext, and $q_l = Q_l/Q_{l-1}$. Let $\text{sk} = (1, s) \in R^2$ be the secret key with hamming weight h . Lets assume $\|\text{Dec}(\hat{\text{ct}})\|_\infty \leq \hat{M}$ and $\|\text{Dec}(\check{\text{ct}})\|_\infty \leq \check{M}$. Also assume that $N\|\text{pt}\|_\infty(\hat{M}q_{\text{div}} + \check{M}) < Q_l/2$. Then the inequality below is satisfied.*

$$\left\| \left[\text{RCB}_{q_{\text{div}}}(\text{PtMult}^2(\text{PT}, \text{CT})) \cdot (1, s) \right]_{Q_{l-1}} - \frac{1}{q_l} \left[\text{RCB}_{q_{\text{div}}}(\text{PT}) \cdot (\text{RCB}_{q_{\text{div}}}(\text{CT}) \cdot (1, s)) \right]_{Q_l} \right\|_\infty \leq \frac{N\check{M}\|\check{\text{pt}}\|_\infty}{q_{\text{div}}q_l} + \frac{h+1}{2}.$$

Proof. By the condition $N\|\text{pt}\|_\infty(\hat{M}q_{\text{div}} + \check{M}) + h < Q_l/2$, we can apply lemma 5, where the term from lemma 5 is divided by q_l in this theorem. We can then combine this with lemma 2 for the Tuple Rescale, which introduces the $\frac{h+1}{2}$ term, leading to the final inequality. $\square$

From now on, Δ_A will represent the scaling factor for plaintext. The overall message scale was divided by q_{div} in the Tuple Tensor step and by q_l in the Tuple Rescale step. Therefore, in the entire Tuple PtMult, the message scale will be divided by $q_{\text{div}}q_l$. As a result, the relation $q_{\text{div}}q_l \approx \Delta_A$ must be satisfied.

In Section 2, we defined the decryption of the remainder part as the lower part. The lower part of the decomposed ciphertext increases when Tuple PtMult is applied. Since the error of Tuple PtMult depends on the size of the lower part, it is crucial to keep track of its size. The theorem below analyzes the bound of the lower part.

Theorem 2. *Let $\text{PT} = (\hat{\text{pt}}, \check{\text{pt}}) \in R_{Q_l} \times R_{Q_l}$, $\text{CT} = (\hat{\text{ct}}, \check{\text{ct}}) \in R_{Q_l}^2 \times R_{Q_l}^2$ be decomposed plaintext and ciphertext, and $q_l = Q_l/Q_{l-1}$. Let $\text{sk} = (1, s) \in R^2$ be the secret key with hamming weight h . Lets assume $\|\text{can} \circ \text{Dec}(\hat{\text{ct}})\|_\infty \leq \hat{M}$ and $\|\text{can} \circ \text{Dec}(\check{\text{ct}})\|_\infty \leq \check{M}$. Let $\text{CT}_{\text{PtMult}} = \text{PtMult}^2(\text{PT}, \text{CT}) = (\hat{\text{ct}}_{\text{PtMult}}, \check{\text{ct}}_{\text{PtMult}})$. Then the inequality below is satisfied.*

$$\|\text{can} \circ \text{Dec}(\check{\text{ct}}_{\text{PtMult}})\|_\infty \leq \frac{\|\text{can}(\check{\text{pt}})\|_\infty \hat{M} + \|\text{can}(\hat{\text{pt}})\|_\infty \check{M}}{q_l} + \frac{N}{2}(h+1)(q_{\text{div}}+1).$$

Proof. Let $\text{Tensor}^2(\text{PT}, \text{CT}) = (\hat{T}, \check{T})$. From the definition of Tensor^2 for PtMult, we can know that

$$\|\text{can}([\check{T} \cdot (1, s)]_{Q_l})\|_\infty \leq \|\text{can}(\check{\text{pt}})\|_\infty \hat{M} + \|\text{can}(\hat{\text{pt}})\|_\infty \check{M}.$$

For some $\hat{e}$ and $\check{e}$ which satisfies $\|\text{can}(\hat{e})\|_\infty \leq \frac{N}{2}$, $\|\text{can}(\check{e})\|_\infty \leq \frac{N}{2}$, the formula below is true,

$$\check{\text{ct}}_{\text{PtMult}} = \frac{1}{q_l}(q_{\text{div}} \cdot \hat{T} + \check{T}) + \check{e} - q_{\text{div}} \left(\frac{1}{q_l} \cdot \hat{T} + \hat{e} \right) = \frac{1}{q_l}\check{T} - q_{\text{div}}\hat{e} + \check{e}.$$

Using the bounds of $\check{T}$, $\hat{e}$, and $\check{e}$, we can get the final inequality of our theorem. $\square$

For the corollary below and for subsequent ones, we define the notation $\check{M}_0 = N \cdot q_{\text{div}} \cdot \frac{h+3}{2}$. From theorem 1, $\check{M}_0$ represents the bound of $\|\text{can} \circ \text{Dec}(\check{\text{ct}})\|_\infty$, where $\check{\text{ct}}$ is the lower part of the decomposed ciphertext that has not undergone any operations.

Corollary 1 (Growth of lower part in Tuple PtMult). *Let $\text{PT} = (\hat{\text{pt}}, \check{\text{pt}}) \in R_{Q_l} \times R_{Q_l}$, $\text{CT} = (\hat{\text{ct}}, \check{\text{ct}}) \in R_{Q_l}^2 \times R_{Q_l}^2$ be decomposed plaintext and ciphertext and $\text{RCB}_{q_{\text{div}}}(\text{PT}) = \text{pt}$, $\text{RCB}_{q_{\text{div}}}(\text{CT}) = \text{ct}$. Let Δ and Δ_A be the scaling factor for ct and pt . Let $\text{sk} = (1, s) \in R^2$ be the secret key with hamming weight h . Lets assume $\|\text{can} \circ \text{Dec}(\hat{\text{ct}})\|_\infty \leq \hat{M}$ and $\|\text{can} \circ \text{Dec}(\check{\text{ct}})\|_\infty \leq \check{M}$. Let $\text{CT}_{\text{PtMult}} = \text{PtMult}^2(\text{PT}, \text{CT}) = (\hat{\text{ct}}_{\text{PtMult}}, \check{\text{ct}}_{\text{PtMult}})$, and $\check{M}_{\text{next}} =$*

$\|\text{can} \circ \text{Dec}(\check{\text{ct}}_{\text{PtMult}})\|_\infty$. Also we will assume $\|\text{Dcd}(\text{pt})\|_\infty \leq 1$, $\|\text{Dcd} \circ \text{Dec}(\text{ct})\|_\infty \leq 1$. Then the inequality below is satisfied.

$$\check{M}_{\text{next}} \leq \check{M} + \check{M}_0 \cdot \left(\frac{1}{h+3} \cdot \frac{\Delta}{\Delta_A} + 1 \right).$$

Proof. First for ct , ($\text{Dcd} = \Delta^{-1} \cdot \text{can}$) holds. So from our assumption $\|\text{Dcd} \circ \text{Dec}(\text{ct})\|_\infty \leq 1$, we can get $\|\text{can} \circ \text{Dec}(\text{ct})\|_\infty \leq \Delta$. Then we get $\hat{M} \leq \frac{\Delta}{q_{\text{div}}}$. For pt , ($\text{Dcd} = \Delta_A^{-1} \cdot \text{can}$) holds, so from our assumption $\|\text{Dcd}(\text{pt})\|_\infty \leq 1$, we can get $\|\text{can}(\text{pt})\|_\infty \leq \Delta_A$. Then we get $\|\text{can}(\hat{\text{pt}})\|_\infty \leq \frac{\Delta_A}{q_{\text{div}}}$.

Now, let's bound the first term on the right-hand side of the previous theorem:

$$\begin{aligned} \frac{\|\text{can}(\check{\text{pt}})\|_\infty \hat{M} + \|\text{can}(\hat{\text{pt}})\|_\infty \check{M}}{q_l} &\leq \frac{1}{q_l} \left(\|\text{can}(\check{\text{pt}})\|_\infty \cdot \frac{\Delta}{q_{\text{div}}} + \frac{\Delta_A}{q_{\text{div}}} \cdot \check{M} \right) \\ &\approx \|\text{can}(\check{\text{pt}})\|_\infty \cdot \frac{\Delta}{\Delta_A} + \check{M}. \end{aligned}$$

Here $\|\text{can}(\check{\text{pt}})\|_\infty \leq \frac{q_{\text{div}}}{2} \cdot N = \check{M}_0 \cdot \frac{1}{h+3}$ holds, so we can get

$$\frac{\|\text{can}(\check{\text{pt}})\|_\infty \hat{M} + \|\text{can}(\hat{\text{pt}})\|_\infty \check{M}}{q_l} \leq \check{M} + \check{M}_0 \cdot \frac{1}{h+3} \cdot \frac{\Delta}{\Delta_A}.$$

We can bound the second term of the right side on the previous theorem as

$$\frac{N}{2} (h+1)(q_{\text{div}}+1) \leq N \cdot q_{\text{div}} \cdot \frac{h+3}{2} = \check{M}_0.$$

By combining the bounds of two terms of the right side on the previous theorem, we get our corollary. $\square$

C.2 Tuple-CKKS on Rot

Definition 8 (Tuple Permute). Let $\text{CT} = (\hat{\text{ct}}, \check{\text{ct}}) \in R_{Q_l}^2 \times R_{Q_l}^2$. And let $\hat{\text{ct}} = (\hat{b}(x), \hat{a}(x))$, $\check{\text{ct}} = (\check{b}(x), \check{a}(x))$. Aut of CT is defined as

$$\text{Aut}_j^2(\text{CT}) = ((\hat{b}(x^{5^j}), \hat{a}(x^{5^j})), (\check{b}(x^{5^j}), \check{a}(x^{5^j}))).$$

Tuple Aut is the exact operation, so there is no error from the Aut step of Rotation in Tuple.

Definition 9 (Tuple Key switching). Let $\text{CT} = (\hat{\text{ct}}, \check{\text{ct}}) \in R_{Q_l}^2 \times R_{Q_l}^2$ be the decomposed ciphertext with secret key $(1, s_1)$. Let's define Tuple KeySwitching from key $(1, s_1)$ to $(1, s_2)$ as

$$\text{Ksw}_{s_1 \rightarrow s_2}^2(\text{CT}) = \text{DCP}_{q_{\text{div}}}(\text{Ksw}_{s_1 \rightarrow s_2}(q_{\text{div}} \cdot \hat{\text{ct}})) + (0, \text{Ksw}_{s_1 \rightarrow s_2}(\check{\text{ct}})).$$

The above definition is almost identical to Relinearization in Tuple, but each Relinearization component is replaced by the corresponding KeySwitching component. In contrast to Relinearization, the input here is in the space $R_{Q_l}^2 \times R_{Q_l}^2$, not $R_{Q_l}^3 \times R_{Q_l}^3$.

Definition 10 (Tuple Rotation). We define Tuple Rotation as $\text{Rot}_j^2 = \text{Ksw}_{s(x^{5^j}) \rightarrow s(x)}^2 \circ \text{Aut}_j^2$. This makes the message to be j times rotated to the left.

From now on, we will use the notation E_{ksw} as the error bound of KeySwitching for normal ciphertext.

Theorem 3 (Error of Tuple Rotation). *Let $\text{CT} = (\hat{\text{ct}}, \check{\text{ct}}) \in R_{Q_l}^2 \times R_{Q_l}^2$ be decomposed ciphertext. Let $\text{sk} = (1, s) \in R^2$ be the secret key with hamming weight h . Let B be the bound for $\| \text{Rot}_j(\text{RCB}_{q_{\text{div}}}(\text{CT})) \|_\infty$. And assume $B + E_{ksw} + h \leq Q_l/4$. Then the inequality below is satisfied.*

$$\| \text{Dec}_{\text{sk}}(\text{RCB}_{q_{\text{div}}}(\text{Rot}_j^2(\text{CT}))) - \text{Dec}_{\text{sk}}(\text{Rot}_j(\text{RCB}_{q_{\text{div}}}(\text{CT}))) \|_\infty \leq E_{ksw} + h.$$

Proof. First we will let $\text{CT}_P = (\hat{\text{ct}}_P, \check{\text{ct}}_P) = \text{Aut}_j^2(\text{CT})$, and Aut is an exact operation so following holds.

$$\text{RCB}_{q_{\text{div}}}(\text{CT}_P) = \text{RCB}_{q_{\text{div}}}(\text{Aut}_j^2(\text{CT})) = \text{Aut}_j(\text{RCB}_{q_{\text{div}}}(\text{CT})).$$

Then by definition of Rot_j^2

$$\begin{aligned} \text{RCB}_{q_{\text{div}}}(\text{Rot}_j^2(\text{CT})) &= \text{RCB}_{q_{\text{div}}}(Ksw_{s(x^{5j}) \rightarrow s(x)}^2 \circ \text{Aut}_j^2(\text{CT})) \\ &= \text{RCB}_{q_{\text{div}}}(Ksw_{s(x^{5j}) \rightarrow s(x)}^2(\text{CT}_P)). \end{aligned}$$

And in modulo Q_l

$$\text{RCB}_{q_{\text{div}}}(Ksw_{s(x^{5j}) \rightarrow s(x)}^2(\text{CT}_P)) = Ksw_{s(x^{5j}) \rightarrow s(x)}(q_{\text{div}} \cdot \hat{\text{ct}}_P + Ksw_{s(x^{5j}) \rightarrow s(x)}(\check{\text{ct}}_P)).$$

Also following holds in modulo Q_l for some $\|e_0\|_\infty \leq 1$.

$$Ksw_{s(x^{5j}) \rightarrow s(x)}(q_{\text{div}} \cdot \hat{\text{ct}}_P + Ksw_{s(x^{5j}) \rightarrow s(x)}(\check{\text{ct}}_P)) = Ksw_{s(x^{5j}) \rightarrow s(x)}(q_{\text{div}} \cdot \hat{\text{ct}}_P + \check{\text{ct}}_P) + (0, e_0).$$

By combining two previous equalities and doing the inner product with $\text{sk} = (1, s)$, we get the following in modulo Q_l .

$$\text{RCB}_{q_{\text{div}}}(Ksw_{s(x^{5j}) \rightarrow s(x)}^2((\text{CT}_P))) \cdot (1, s) = Ksw_{s(x^{5j}) \rightarrow s(x)}(q_{\text{div}} \cdot \hat{\text{ct}}_P + \check{\text{ct}}_P) \cdot (1, s) + e_0 \cdot s.$$

And E_{ksw} is KeySwitching error bound, so

$$\| [Ksw_{s(x^{5j}) \rightarrow s(x)}(q_{\text{div}} \cdot \hat{\text{ct}}_P + \check{\text{ct}}_P) \cdot (1, s) - (q_{\text{div}} \cdot \hat{\text{ct}}_P + \check{\text{ct}}_P) \cdot (1, s(x^{5j}))]_{Q_l} \|_\infty \leq E_{ksw}.$$

The following holds.

$$\begin{aligned} & \| [\text{RCB}_{q_{\text{div}}}(\text{Rot}_j^2(\text{CT})) \cdot (1, s) - \text{Rot}_j(\text{RCB}_{q_{\text{div}}}(\text{CT})) \cdot (1, s)]_{Q_l} \|_\infty \\ &= \| [\text{RCB}_{q_{\text{div}}}(Ksw_{s(x^{5j}) \rightarrow s(x)}^2(\text{CT}_P)) \cdot (1, s) - \text{Rot}_j(\text{RCB}_{q_{\text{div}}}(\text{CT})) \cdot (1, s)]_{Q_l} \|_\infty \\ &= \| [Ksw_{s(x^{5j}) \rightarrow s(x)}(q_{\text{div}} \cdot \hat{\text{ct}}_P + \check{\text{ct}}_P) \cdot (1, s) \\ &\quad + e_0 \cdot s - Ksw_{s(x^{5j}) \rightarrow s(x)} \circ \text{Aut}_j(\text{RCB}_{q_{\text{div}}}(\text{CT})) \cdot (1, s)]_{Q_l} \|_\infty \\ &= \| [Ksw_{s(x^{5j}) \rightarrow s(x)}(q_{\text{div}} \cdot \hat{\text{ct}}_P + \check{\text{ct}}_P) \cdot (1, s) \\ &\quad + e_0 \cdot s - Ksw_{s(x^{5j}) \rightarrow s(x)}(\text{RCB}_{q_{\text{div}}}(\text{CT}_P)) \cdot (1, s)]_{Q_l} \|_\infty \\ &\leq \| [Ksw_{s(x^{5j}) \rightarrow s(x)}(q_{\text{div}} \cdot \hat{\text{ct}}_P + \check{\text{ct}}_P) \cdot (1, s) \\ &\quad - (q_{\text{div}} \cdot \hat{\text{ct}}_P + \check{\text{ct}}_P) \cdot (1, s(x^{5j}))]_{Q_l} \|_\infty + \|e_0 \cdot s\|_\infty \\ &\leq E_{ksw} + h. \end{aligned}$$

And the message on coefficients will be smaller than $\frac{Q_l}{4}$, so the following holds.

$$\begin{aligned} & \| \text{Dec}_{\text{sk}}(\text{RCB}_{q_{\text{div}}}(\text{Rot}_j^2(\text{CT}))) - \text{Dec}_{\text{sk}}(\text{Rot}_j(\text{RCB}_{q_{\text{div}}}(\text{CT}))) \|_\infty \\ &= \| [\text{RCB}_{q_{\text{div}}}(\text{Rot}_j^2(\text{CT})) \cdot (1, s) - \text{Rot}_j(\text{RCB}_{q_{\text{div}}}(\text{CT})) \cdot (1, s)]_{Q_l} \|_\infty \\ &\leq E_{ksw} + h. \end{aligned}$$

□

Theorem 4 (Growth of lower part in Tuple Rotation). *Let $\text{CT} = (\hat{\text{ct}}, \check{\text{ct}}) \in R_{Q_l}^2 \times R_{Q_l}^2$ be a decomposed ciphertext, and let $\text{sk} = (1, s) \in R^2$ be the secret key with Hamming weight h . Assume that $\|\text{can} \circ \text{Dec}(\hat{\text{ct}})\|_\infty \leq \check{M}$ and $\|\text{can} \circ \text{Dec}(\check{\text{ct}})\|_\infty \leq \check{M}$. Let $\text{CT}_{\text{Rot}} = \text{Rot}_j^2(\text{CT}) = (\hat{\text{ct}}_{\text{Rot}}, \check{\text{ct}}_{\text{Rot}})$. Let $\check{M}_{\text{next}} = \|\text{can} \circ \text{Dec}(\check{\text{ct}}_{\text{Rot}})\|_\infty$. Then the following inequality holds:*

$$\check{M}_{\text{next}} \leq \check{M} + \check{M}_0 + N \cdot E_{ksw}.$$

Proof. Let $\text{CT}_P = (\hat{\text{ct}}_P, \check{\text{ct}}_P) = \text{Aut}_j^2(\text{CT})$. The following holds:

$$(\hat{\text{ct}}_{\text{Rot}}, \check{\text{ct}}_{\text{Rot}}) = \text{Rot}_j^2(\text{CT}) = Ksw_{s(x^{5j}) \rightarrow s(x)}^2(\text{Aut}_j^2(\text{CT})) = Ksw_{s(x^{5j}) \rightarrow s(x)}^2(\text{CT}_P).$$

And by definition of Tuple Key-Switching,

$$(\hat{\text{ct}}_{\text{Rot}}, \check{\text{ct}}_{\text{Rot}}) = \text{DCP}_{q_{\text{div}}}(Ksw_{s(x^{5j}) \rightarrow s(x)}(q_{\text{div}} \cdot \hat{\text{ct}}_P)) + (0, Ksw_{s(x^{5j}) \rightarrow s(x)}(\check{\text{ct}}_P)).$$

For $\text{DCP}_{q_{\text{div}}}(Ksw_{s(x^{5j}) \rightarrow s(x)}(q_{\text{div}} \cdot \hat{\text{ct}}_P))$, the lower part will be bounded by the initial lower part bound $\check{M}_0$. For $(0, Ksw_{s(x^{5j}) \rightarrow s(x)}(\check{\text{ct}}_P))$, while Aut does not affect the lower part bound, $\check{\text{ct}}_P$ will have the bound $\check{M}$, and the term NE_{ksw} will be added due to the error from Key-Switching. By combining the analyses of these two terms, we obtain our theorem. $\square$